

计算机网络 | 大作业报告

学号：22070001035

姓名：洪佳荣

专业：软件工程

年级：2022 级

一、实验过程

实验基于老师提供的 RDT 1.0 代码，使用 IntelliJ IDEA 进行开发，Java 版本为 1.8。使用 Git 进行版本控制实现代码的迭代开发。

1. RDT 1.0（原始版本）

配置好开发环境后，首先运行老师提供的 RDT 1.0 代码，观察其运行结果。RDT 1.0 代码实现了一个简单的 TCP 发送端和接收端，通过控制台输出发送端和接收端的状态信息。由于 RDT 1.0 代码基于所依赖的信道十分可靠的假设，因此对于数据包的位错、丢失或者重复等情况并没有进行处理。

设置 `tcpH.setTh_eflag((byte) 0)`，即关闭信道的错误标志，运行 RDT 1.0 代码。

984	2024-12-18 11:42:34:454 CST DATA_seq: 98101	ACKed	1985
985	2024-12-18 11:42:34:467 CST DATA_seq: 98201	ACKed	1986
986	2024-12-18 11:42:34:481 CST DATA_seq: 98301	ACKed	1987
987	2024-12-18 11:42:34:492 CST DATA_seq: 98401	ACKed	1988
988	2024-12-18 11:42:34:506 CST DATA_seq: 98501	ACKed	1989
989	2024-12-18 11:42:34:519 CST DATA_seq: 98601	ACKed	1990
990	2024-12-18 11:42:34:532 CST DATA_seq: 98701	ACKed	1991
991	2024-12-18 11:42:34:546 CST DATA_seq: 98801	ACKed	1992
992	2024-12-18 11:42:34:558 CST DATA_seq: 98901	ACKed	1993
993	2024-12-18 11:42:34:570 CST DATA_seq: 99001	ACKed	1994
994	2024-12-18 11:42:34:583 CST DATA_seq: 99101	ACKed	1995
995	2024-12-18 11:42:34:597 CST DATA_seq: 99201	ACKed	1996
996	2024-12-18 11:42:34:612 CST DATA_seq: 99301	ACKed	1997
997	2024-12-18 11:42:34:640 CST DATA_seq: 99401	ACKed	1998
998	2024-12-18 11:42:34:653 CST DATA_seq: 99501	ACKed	1999
999	2024-12-18 11:42:34:665 CST DATA_seq: 99601	ACKed	2000
1000	2024-12-18 11:42:34:679 CST DATA_seq: 99701	ACKed	2001
1001	2024-12-18 11:42:34:692 CST DATA_seq: 99801	ACKed	2002
1002	2024-12-18 11:42:34:705 CST DATA_seq: 99901	ACKed	2003

图 1 Sender Log

2024-12-18 11:42:34:441 CST ACK_ack: 98001	99980	1299377
2024-12-18 11:42:34:454 CST ACK_ack: 98101	99981	1299379
2024-12-18 11:42:34:468 CST ACK_ack: 98201	99982	1299437
2024-12-18 11:42:34:481 CST ACK_ack: 98301	99983	1299439
2024-12-18 11:42:34:493 CST ACK_ack: 98401	99984	1299449
2024-12-18 11:42:34:506 CST ACK_ack: 98501	99985	1299451
2024-12-18 11:42:34:519 CST ACK_ack: 98601	99986	1299457
2024-12-18 11:42:34:533 CST ACK_ack: 98701	99987	1299491
2024-12-18 11:42:34:546 CST ACK_ack: 98801	99988	1299499
2024-12-18 11:42:34:559 CST ACK_ack: 98901	99989	1299533
2024-12-18 11:42:34:571 CST ACK_ack: 99001	99990	1299541
2024-12-18 11:42:34:584 CST ACK_ack: 99101	99991	1299553
2024-12-18 11:42:34:598 CST ACK_ack: 99201	99992	1299583
2024-12-18 11:42:34:612 CST ACK_ack: 99301	99993	1299601
2024-12-18 11:42:34:640 CST ACK_ack: 99401	99994	1299631
2024-12-18 11:42:34:653 CST ACK_ack: 99501	99995	1299637
2024-12-18 11:42:34:666 CST ACK_ack: 99601	99996	1299647
2024-12-18 11:42:34:679 CST ACK_ack: 99701	99997	1299653
2024-12-18 11:42:34:692 CST ACK_ack: 99801	99998	1299673
2024-12-18 11:42:34:706 CST ACK_ack: 99901	99999	1299689
	100000	1299709

图 2 Receiver Log

图 3 recvData

对于发送端所有发出的包，接收端都能够正确接收并返回确认，recvData 中的数据也是完整的。

2. RDT 2.x

RDT 2.0

RDT 2.0 假设信道是不可靠的，即数据包可能会出现位错的情况。因此，RDT 2.0 需要在发送端和接收端实现数据包的校验和功能，以便在接收端检测出数据包是否出现位错。

```
1 public static short computeChkSum(TCP_PACKET tcpPack) {
2     TCP_HEADER tcpHeader = tcpPack.getTcpH();
3     int seq = tcpHeader.getTh_seq();
4     int ack = tcpHeader.getTh_ack();
5     TCP_SEGMENT tcpSegment = tcpPack.getTcpS();
6     int[] data = tcpSegment.getData();
7
8     CRC32 crc32 = new CRC32();
9     crc32.update(seq);
10    crc32.update(ack);
11    for (int i : data) {
12        crc32.update(i);
13    }
14 }
```

```

14
15     int checksum = (int) crc32.getValue();
16     return (short) checksum;
17 }

```

对传入的数据包进行校验和计算，分别将其序列号、确认号和数据部分的每个字节进行 CRC32 计算，得到校验和。这里有一个出入，老师的 PPT 上要求“校验和”也参与计算，但是我认为校验和不应该参与计算，因为校验和是用来校验数据包是否出错的，如果校验和也参与计算，那么校验和的值就会影响校验和的计算结果。

在实现校验和方法前，所有包的校验和均为 0，即不进行校验和计算。

RDT 2.1 / RDT 2.2

对于 RDT 2.1，接收方按序依次接收数据包，如果接收到的数据包的校验和正确，则先返回 ACK，然后查看数据包的序列号是否为期望的序列号，如果不是则不做处理，如果是则将数据包的放进缓冲以等待交付。而如果校验和错误，则返回一个 NAK。对于 RDT 2.2，接收方不使用 NAK，而是重传对上一个正确接收的数据包的 ACK。

实现上，注意到数据包的序列号 seq 是字节流号，而原来设计中接收方的 sequence 是一个递增的值，可以认为就是数据包的序号，因此需要将发送方发来的 seq 转换为 dataIndex。在 TCP_Sender 中，seq 的计算公式是

$$\text{seq} = \text{dataIndex} * \text{dataLength} + 1,$$

所以反过来计算 dataIndex 的公式是

$$\text{dataIndex} = \frac{\text{seq} - 1}{\text{dataLength}}.$$

这样，要重传上一个正确接收的数据包的 ACK，只需要将上一个正确接收的数据包的 dataIndex（即当前期望的 dataIndex 减 1）算出来即可。在代码中，使用和老师设计一致的 sequence 来表示当前期望的 dataIndex。

在 TCP_Receiver 中，作出如下修改（黄色高亮部分）：

```

1  int sequence = 0; // 用于记录当前待接收的包序号，将原来的 1 改为 0，因为第一个包的序号是 0
2
3  // ...
4
5  @Override
6  // 接收到数据报：检查校验和，设置回复的 ACK 报文段
7  public void rdt_recv(TCP_PACKET recvPack) {
8      // 检查校验码，生成 ACK
9      int dataLength = recvPack.getTcpS().getData().length;
10     int dataSequence = (recvPack.getTcpH().getTh_seq() - 1) / dataLength;
11     if (Checksum.computeChkSum(recvPack) == recvPack.getTcpH().getTh_sum()) {
12         // 生成 ACK 报文段（设置确认号）
13         tcpH.setTh_ack(recvPack.getTcpH().getTh_seq());
14         ackPack = new TCP_PACKET(tcpH, tcpS, recvPack.getSourceAddr());
15         tcpH.setTh_sum(Checksum.computeChkSum(ackPack));
16         // 回复 ACK 报文段
17         reply(ackPack);
18
19         // 将接收到的正确有序的数据插入 data 队列，准备交付
20         if (dataSequence == sequence) {
21             dataQueue.add(recvPack.getTcpS().getData());

```

```

22         sequence++;
23     }
24 } else {
25     System.out.println("Recieve Computed: " + CheckSum.computeChkSum(recvPack));
26     System.out.println("Recieved Packet" + recvPack.getTcpH().getTh_sum());
27     System.out.println("Problem: Packet Number: " + recvPack.getTcpH().getTh_seq()
+ " + InnerSeq: " + sequence);
28     tcpH.setTh_ack((sequence - 1) * dataLength + 1); // 无需使用 NAK
29     ackPack = new TCP_PACKET(tcpH, tcpS, recvPack.getSourceAddr());
30     tcpH.setTh_sum(CheckSum.computeChkSum(ackPack));
31     // 回复 ACK 报文段
32     reply(ackPack);
33 }
34
35 System.out.println();
36
37 // 交付数据 (每 20 组数据交付一次)
38 if (dataQueue.size() == 20)
39     deliver_data();
40 }

```

先根据接收到的数据包计算出其 `dataIndex` (`dataSequence`, 保持和一开始的变量名一致), 然后检查校验和, 如果校验和正确, 则该包没有出现位错, 生成 ACK 报文段, 设置确认号为接收到的数据包的序列号, 然后回复 ACK 报文段。如果校验和错误, 则生成 ACK 报文段, 设置确认号为上一个正确接收的数据包的序列号, 然后回复 ACK 报文段。

没有出现位错并不意味着这个包一定是有序的期望的包, 因此需要判断接收到的包的 `dataIndex` 是否为期望的 `dataIndex` (比较 `dataSequence` 和 `sequence`), 如果是则交付数据, 放入 `dataQueue` 中, `sequence` 加 1。

而对于发送方, 老师提供的代码中已经实现了收到错误的 ACK 包后重传数据包的功能, 因此不需要做额外的修改。

将接收方和发送方设置为 `tcpH.setTh_eflag((byte) 1)`, 允许信道出错, 运行 RDT 2.2 代码。

57	2024-12-18 18:54:14:537 CST DATA_seq: 5301	ACKed
58	2024-12-18 18:54:14:548 CST DATA_seq: 5401	ACKed
59	2024-12-18 18:54:14:562 CST DATA_seq: 5501	ACKed
60	2024-12-18 18:54:14:573 CST DATA_seq: 5601	WRONG NO_ACK
61	2024-12-18 18:54:14:574 CST *Re: DATA_seq: 5601	ACKed
62	2024-12-18 18:54:14:587 CST DATA_seq: 5701	ACKed
63	2024-12-18 18:54:14:601 CST DATA_seq: 5801	ACKed
64	2024-12-18 18:54:14:614 CST DATA_seq: 5901	ACKed

图 4 Sender Log

在发送方的 Log 中可以看到, 发送方发送了一个错误的数据包 5601 后没有收到对应的 ACK, 显示 NO_ACK, 然后重传了数据包 5601。

1075	2024-12-18 18:54:14:523 CST ACK_ack: 5201
1076	2024-12-18 18:54:14:537 CST ACK_ack: 5301
1077	2024-12-18 18:54:14:549 CST ACK_ack: 5401
1078	2024-12-18 18:54:14:562 CST ACK_ack: 5501
1079	2024-12-18 18:54:14:574 CST ACK_ack: 5501
1080	2024-12-18 18:54:14:574 CST ACK_ack: 5601
1081	2024-12-18 18:54:14:588 CST ACK_ack: 5701
1082	2024-12-18 18:54:14:601 CST ACK_ack: 5801

图 5 Receiver Log

查看接收方 Log 可以看到这个错误的数据包 5601 被接收方检测出校验和错误，因此接收方返回了上一个 ACK 即 5501。发送方收到重发的 ACK 后重传了数据包 5601。

1200	2024-12-18 18:54:16:126 CST ACK_ack: 17601	99984	1299449
1201	2024-12-18 18:54:16:139 CST ACK_ack: 17701	99985	1299451
1202	2024-12-18 18:54:16:150 CST ACK_ack: 17801	99986	1299457
1203	2024-12-18 18:54:16:162 CST ACK_ack: -629701288 WRONG	99987	1299491
1204	2024-12-18 18:54:16:163 CST ACK_ack: 17901	99988	1299499
1205	2024-12-18 18:54:16:174 CST ACK_ack: 18001	99989	1299533
1206	2024-12-18 18:54:16:188 CST ACK_ack: 18101	99990	1299541
1207	2024-12-18 18:54:16:201 CST ACK_ack: 18201	99991	1299553

图 6 Receiver Log

181	2024-12-18 18:54:16:126 CST DATA_seq: 17601	ACKed	99994	1299631
182	2024-12-18 18:54:16:139 CST DATA_seq: 17701	ACKed	99995	1299637
183	2024-12-18 18:54:16:150 CST DATA_seq: 17801	ACKed	99996	1299647
184	2024-12-18 18:54:16:162 CST DATA_seq: 17901	NO_ACK	99997	1299653
185	2024-12-18 18:54:16:162 CST *Re: DATA_seq: 17901	ACKed	99998	1299673
186	2024-12-18 18:54:16:174 CST DATA_seq: 18001	ACKed	99999	1299689
187	2024-12-18 18:54:16:187 CST DATA_seq: 18101	ACKed	100000	1299709
188	2024-12-18 18:54:16:201 CST DATA_seq: 18201	ACKed		

图 7 Sender Log

图 8 recvData

而在接收方这里的 Log 中，可以看到接收方发送了一个错误的 ACK，根据前一个 ACK 是 17801 可以判断这里本应该发送 17901 的 ACK。发送方收到了错误的 ACK 后，重传了数据包 17901。于是接收方又一次收到了正确的数据包，发送了正确的 ACK 并交付数据。

从图 8 可以看到，接收方收到的数据包是有序的且数量为 100000，没有出现数据包的丢失或者重复。

3. RDT 3.0

RDT 3.0 在 RDT 2.2 的基础上作出了新的假设，即数据包可能会出现丢失的情况。因此，RDT 3.0 需要在发送方实现超时重传的功能，即发送方发送数据包后，如果在一定时间内没有收到 ACK，则重传数据包。此时就不再需要接收方重传 ACK，接收方直接丢弃不在期望范围内的数据包并不做响应。

在 TCP_Sender 类中，作出如下修改（黄色高亮部分）：

```

1 // 计时器
2 private UDT_Timer timer;
3
4 @Override
5 // 可靠发送（应用层调用）：封装应用层数据，产生 TCP 数据报；需要修改
6 public void rdt_send(int dataIndex, int[] appData) {
7     // 生成 TCP 数据报（设置序号和数据字段/校验和），注意打包的顺序
8     tcpH.setTh_seq(dataIndex * appData.length + 1); // 包序号设置为字节流号：
9     tcpS.setData(appData);
10    tcpPack = new TCP_PACKET(tcpH, tcpS, destinAddr);
11    // 更新带有 checksum 的 TCP 报文头
12    tcpH.setTh_sum(CheckSum.computeChkSum(tcpPack));
13    tcpPack.setTcpH(tcpH);
14
15    // 对每个数据包设置定时器
16    // 重传任务
17    timer = new UDT_Timer();
18    UDT_RetransTask retransTask = new UDT_RetransTask(client, tcpPack);
19    timer.schedule(retransTask, 1000, 1000); // 1s 后开始重传，每 1s 重传一次
20

```

```

21 // 发送 TCP 数据报
22 udt_send(tcpPack);
23 flag = 0;
24
25 // 等待 ACK 报文
26 // waitACK();
27 while (flag == 0)
28     ;
29 }
30
31 // ...
32
33 @Override
34 public void waitACK() {
35     // 循环检查 ackQueue
36     // 循环检查确认号对列中是否有新收到的 ACK
37     if (!ackQueue.isEmpty()) {
38         int currentAck = ackQueue.poll();
39         // 接收到的 ACK 号等于当前发送的包号, 说明发送成功
40         if (currentAck == tcpPack.getTcpH().getTh_seq()) {
41             timer.cancel(); // 取消定时器
42             System.out.println("Clear: " + tcpPack.getTcpH().getTh_seq());
43             flag = 1;
44             // break;
45         }
46         // 不等于, 说明发送失败, 由计时器触发重传
47     }
48 }

```

在发送方中, 使用 `Timer` (即我这里使用的老师封装好的 `UDT_Timer`) 和 `TimerTask` 来实现超时重传的功能。在发送数据包后, 启动一个定时器, 在这里我设置定时器每隔 1s 重传数据包。在等待 ACK 的过程中, 如果收到了 ACK, 则取消定时器, 否则在定时器触发时重传数据包。

在 `TCP_Receiver` 中, 只修改了接收方的代码, 将接收方的重传 ACK 的功能去掉, 直接丢弃不在期望范围内的数据包。如下:

```

1 if (Checksum.computeChkSum(recvPack) == recvPack.getTcpH().getTh_sum()) {
2     // 如果接收到的数据包的序号小于等于期望的序号, 则接收数据, 可能是重传的数据包
3     if (dataSequence <= sequence) {
4         // 生成 ACK 报文段 (设置确认号)
5         tcpH.setTh_ack(recvPack.getTcpH().getTh_seq());
6         ackPack = new TCP_PACKET(tcpH, tcpS, recvPack.getSourceAddr());
7         tcpH.setTh_sum(Checksum.computeChkSum(ackPack));
8         // 回复 ACK 报文段
9         reply(ackPack);
10
11         // 如果接收到的数据包的序号等于期望的序号, 则将数据包放进缓冲以等待交付
12         if (dataSequence == sequence) {
13             sequence = dataSequence + 1;
14             dataQueue.add(recvPack.getTcpS().getData());
15         }
16     }
17 } else {
18     // ...
19 }

```

和原来相比，只加了两个判断条件，一个是判断接收到的数据包的序号是否小于等于期望的序号，如果是则接收数据包并回复 ACK；另一个是判断接收到的数据包的序号是否等于期望的序号，如果是则交付数据。对于超前的数据包，直接丢弃。出错的数据包也一样，直接丢弃且不回复 ACK。

这样设计也可以处理延迟送达的包。当一个数据包延迟送达时，发送方无法收到 ACK，会在定时器触发时重传数据包，和丢包的情况一样。所以此时可以使用 `tcpH.setTh_eflag((byte) 7)`，允许信道出错、丢包、延迟，运行 RDT 3.0 代码。

629	2024-12-18 20:13:18:238 CST DATA_seq: 61601	ACKed
630	2024-12-18 20:13:18:259 CST DATA_seq: 61701	ACKed
631	2024-12-18 20:13:18:272 CST DATA_seq: 61801	ACKed
632	2024-12-18 20:13:18:285 CST DATA_seq: 61901	WRONG NO_ACK
633	2024-12-18 20:13:19:290 CST *Re: DATA_seq: 61901	ACKed
634	2024-12-18 20:13:19:303 CST DATA_seq: 62001	ACKed
635	2024-12-18 20:13:19:316 CST DATA_seq: 62101	ACKed
636	2024-12-18 20:13:19:329 CST DATA_seq: 62201	ACKed

图 9 Sender Log

在发送方的 Log 中可以看到，发送方发送了一个错误的数据包 61901 后没有收到对应的 ACK，显示 NO_ACK，然后重传了数据包 61901。

1644	2024-12-18 20:13:18:238 CST ACK_ack: 61601	
1645	2024-12-18 20:13:18:259 CST ACK_ack: 61701	
1646	2024-12-18 20:13:18:272 CST ACK_ack: 61801	
1647	2024-12-18 20:13:19:290 CST ACK_ack: 61901	
1648	2024-12-18 20:13:19:304 CST ACK_ack: 62001	
1649	2024-12-18 20:13:19:317 CST ACK_ack: 62101	
1650	2024-12-18 20:13:19:330 CST ACK_ack: 62201	

图 10 Receiver Log

查看接收方 Log 的时间，可以看到对于前一个错误包，接收方没有进行确认，而过了一会儿等到后面重发的包 61901 时，接收方才进行了确认，发送方才继续发送后面的数据。

1183	2024-12-18 20:13:03:110 CST ACK_ack: 15801	
1184	2024-12-18 20:13:03:123 CST ACK_ack: 15901	
1185	2024-12-18 20:13:03:136 CST ACK_ack: 16001	
1186	2024-12-18 20:13:03:150 CST ACK_ack: 16101	DELAY
1187	2024-12-18 20:13:04:155 CST *Re: ACK_ack: 16101	
1188	2024-12-18 20:13:04:168 CST ACK_ack: 16201	
1189	2024-12-18 20:13:04:181 CST ACK_ack: 16301	
1190	2024-12-18 20:13:04:194 CST ACK_ack: 16401	

图 11 Receiver Log

再比如接收方发送了一个延迟的 16101 确认。

162	2024-12-18 20:13:03:109 CST DATA_seq: 15801	ACKed
163	2024-12-18 20:13:03:122 CST DATA_seq: 15901	ACKed
164	2024-12-18 20:13:03:136 CST DATA_seq: 16001	ACKed
165	2024-12-18 20:13:03:149 CST DATA_seq: 16101	NO_ACK
166	2024-12-18 20:13:04:154 CST *Re: DATA_seq: 16101	ACKed
167	2024-12-18 20:13:04:168 CST DATA_seq: 16201	ACKed
168	2024-12-18 20:13:04:181 CST DATA_seq: 16301	ACKed
169	2024-12-18 20:13:04:194 CST DATA_seq: 16401	ACKed

图 12 Sender Log

发送方计时器超时后发现没有收到期望的 ACK，于是将 16101 对应的数据包重发，此时接收方收到了这个包才重发了 ACK。

470	2024-12-18 20:13:09:196 CST DATA_seq: 46401 ACKed
471	2024-12-18 20:13:09:209 CST DATA_seq: 46501 LOSS NO_ACK
472	2024-12-18 20:13:10:209 CST *Re: DATA_seq: 46501 ACKed
473	2024-12-18 20:13:10:222 CST DATA_seq: 46601 ACKed
474	2024-12-18 20:13:10:235 CST DATA_seq: 46701 LOSS NO_ACK
475	2024-12-18 20:13:11:240 CST *Re: DATA_seq: 46701 ACKed
476	2024-12-18 20:13:11:253 CST DATA_seq: 46801 ACKed
477	2024-12-18 20:13:11:266 CST DATA_seq: 46901 ACKed
478	2024-12-18 20:13:11:279 CST DATA_seq: 47001 ACKed
479	2024-12-18 20:13:11:293 CST DATA_seq: 47101 ACKed
480	2024-12-18 20:13:11:306 CST DATA_seq: 47201 ACKed
481	2024-12-18 20:13:11:319 CST DATA_seq: 47301 DELAY NO_ACK
482	2024-12-18 20:13:12:324 CST *Re: DATA_seq: 47301 ACKed

图 13 Sender Log

对于发送方发送的延迟的包（图中的 47301），在测试系统中会在全部包发完之后才发送，自然在计时器时间范围内，接收方不会收到期望的包，在发送过程中和丢失（如图中 46501、46701）没有什么区别。所以计时器时间到时，发送方便会再发一次，此时接收方收到包没有异常就会回复一个确认了。

再比如接收方的 ACK 出错的情况：

1044	2024-12-18 20:13:00:277 CST ACK_ack: 2001
1045	2024-12-18 20:13:00:290 CST ACK_ack: 2101
1046	2024-12-18 20:13:00:304 CST ACK_ack: 2201
1047	2024-12-18 20:13:00:316 CST ACK_ack: -1413143737 WRONG
1048	2024-12-18 20:13:01:319 CST ACK_ack: 2301
1049	2024-12-18 20:13:01:333 CST ACK_ack: 2401
1050	2024-12-18 20:13:01:346 CST ACK_ack: 2501
1051	2024-12-18 20:13:01:360 CST ACK_ack: 2601

图 14 Receiver Log

接收方发送了一个错误的 ACK，发送方没有收到期望的 ACK，于是重发了数据包。

23	2024-12-18 20:13:00:276 CST DATA_seq: 2001 ACKed
24	2024-12-18 20:13:00:290 CST DATA_seq: 2101 ACKed
25	2024-12-18 20:13:00:304 CST DATA_seq: 2201 ACKed
26	2024-12-18 20:13:00:316 CST DATA_seq: 2301 NO_ACK
27	2024-12-18 20:13:01:318 CST *Re: DATA_seq: 2301 ACKed
28	2024-12-18 20:13:01:332 CST DATA_seq: 2401 ACKed
29	2024-12-18 20:13:01:346 CST DATA_seq: 2501 ACKed
30	2024-12-18 20:13:01:360 CST DATA_seq: 2601 ACKed

图 15 Sender Log

发送方重发了数据包，接收方收到了数据包后发送了正确的 ACK。以及接收方 ACK 延迟的情况：

1184	2024-12-18 20:13:03:123 CST ACK_ack: 15901
1185	2024-12-18 20:13:03:136 CST ACK_ack: 16001
1186	2024-12-18 20:13:03:150 CST ACK_ack: 16101 DELAY
1187	2024-12-18 20:13:04:155 CST *Re: ACK_ack: 16101
1188	2024-12-18 20:13:04:168 CST ACK_ack: 16201
1189	2024-12-18 20:13:04:181 CST ACK_ack: 16301
1190	2024-12-18 20:13:04:194 CST ACK_ack: 16401

图 16 Receiver Log

接收方发送了一个延迟的 ACK，观察时间，可以看到发送方在计时器超时后重发了数据包。

162	2024-12-18 20:13:03:109 CST DATA_seq: 15801	ACKed
163	2024-12-18 20:13:03:122 CST DATA_seq: 15901	ACKed
164	2024-12-18 20:13:03:136 CST DATA_seq: 16001	ACKed
165	2024-12-18 20:13:03:149 CST DATA_seq: 16101	NO_ACK
166	2024-12-18 20:13:04:154 CST *Re: DATA_seq: 16101	ACKed
167	2024-12-18 20:13:04:168 CST DATA_seq: 16201	ACKed
168	2024-12-18 20:13:04:181 CST DATA_seq: 16301	ACKed
169	2024-12-18 20:13:04:194 CST DATA_seq: 16401	ACKed

图 17 Sender Log

综上，对于 RDT 3.0 的关注点实现了以下几点：

- 出错
 - 数据包出错
 - ✓ 检查 Log 文件，是否所有出错的数据包都进行了重传，并且重传的数据包都得到了确认；检查输出文件，是否对应包的正确内容包含在输出文件中，而不是包含出错的包内容以及重复的包内容（重复可以通过检查输出文件行数的办法来看）。
 - ✓ 检查收方在收到错误的数据包时，是否发错的是对上一个序号的确认。
 - ✓ 发方在收到上一个序号的确认时，是否重传当前数据包。
 - 确认包出错
 - ✓ 检查 Log 文件，出错的确认包是否引发了当前包的重传，检查输出文件，是否对应包的正确内容包含在输出文件中，而且不包含出错的包内容以及重复的包内容。
 - ✓ 检查发送方收到错误的确认时，是否重传当前包。
- 丢失
 - ✓ 检查 Log 文件，发生丢包（包含数据包和确认包），是否发送方在 3s 后进行了超时重传（此处实现是 1s）。检查输出文件，是否对应包的正确内容包含在输出文件中，而且不包含重复的包内容（重复可以通过检查输出文件行数的办法来看）。

接下来实现滑动窗口类型的几个协议：Go-Back-N、Select Response 和 TCP。相比较前面的几个协议，使用滑动窗口的协议无需逐个包等待 ACK，而是流水线式地发送数据包，提高了传输效率。我先实现的是 Select Response 协议。

4. Select Response

对于选择响应协议，发送方和接收方各维护一个窗口。发送方的窗口存放了所有准备发送或者已经发送但是还没有收到 ACK 的分组，接收方的窗口存放了所有准备接收或者已经接收但是还没有交付的分组。收发数据时，接收方逐个对所有正确收到的分组进行应答、对接收到的分组进行缓存，当左沿指针指向的包到达时对上层进行有效递交。发送方为每个分组维护一个计时器，当计时器超时时重传对应的分组。

图 18 是发送方窗口的 UML 类图。在 SR 中我使用的数据结构是以数组形式实现的循环队列。队列的最大大小是窗口大小，队列中的每个元素是一个封装好的数据类型 SenderElem，包含了数据包、计时器和其 ACK 状态（用枚举类型表示）。

发送方窗口维护 3 个指针：base、nextToSend 和 rear。base 指向窗口的左沿，是整个队列中第一个未被 ACK 的包；nextToSend 指向下一个要发送的包；rear 指向队列中新加入的包之后。

在发送数据包前，先使用 `pushPacket` 方法将数据包加入队列，然后使用 `sendPacket` 方法发送数据包并启动计时器。当收到 ACK 后，使用 `ackPacket` 方法找到对应的包并更新其 ACK 状态，然后检查是否可以滑动窗口。

当窗口满时，发送方无法将新的包加入队列，直到窗口中的包被 ACK。

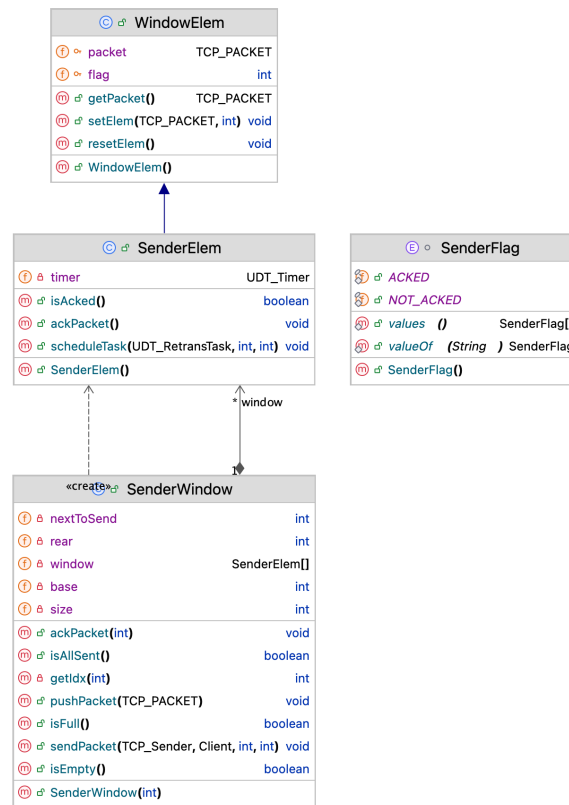


图 18 SenderWindow UML 类图

具体代码实现上，使用数组实现循环队列会显得代码量稍微多一些，主要原因还是要计算下标。

- WindowElem:

WindowElem 是 SenderElem 和 ReceiverElem 的父类，封装了数据包和其 ACK 状态。

```

1 package com.ouc.tcp.test;
2
3 import com.ouc.tcp.message.TCP_PACKET;
4
5 public class WindowElem {
6     protected TCP_PACKET packet;
7     protected int flag;
8
9     public WindowElem() {
10         packet = null;
11         flag = 0;
12     }
13
14     public TCP_PACKET getPacket() {
15         return packet;
16     }
17
18     public void setElem(TCP_PACKET packet, int flag) {
19         this.packet = packet;
20     }
21 }
  
```

```

20         this.flag = flag;
21     }
22
23     public void resetElem() {
24         packet = null;
25         flag = 0;
26     }
27 }
28

```

- SenderElem:

SenderElem 类在 WindowElem 的基础上增加了计时器的功能。计时器的实现和 RDT 3.0 中的计时器一样，使用 Timer 和 TimerTask 来实现。

```

1 package com.ouc.tcp.test;
2
3 import com.ouc.tcp.client.UDT_RetransTask;
4 import com.ouc.tcp.client.UDT_Timer;
5
6 enum SenderFlag {
7     NOT_ACKED, ACKED
8 }
9
10 public class SenderElem extends WindowElem{
11     private UDT_Timer timer;
12
13     public SenderElem() {
14         super();
15         this.timer = null;
16     }
17
18     public boolean isAked() {
19         return flag == SenderFlag.ACKED.ordinal();
20     }
21
22     public void scheduleTask(UDT_RetransTask retransTask, int delay, int period) {
23         this.timer = new UDT_Timer();
24         this.timer.schedule(retransTask, delay, period);
25     }
26
27     public void ackPacket() {
28         this.flag = SenderFlag.ACKED.ordinal();
29         this.timer.cancel();
30     }
31 }

```

- SenderWindow:

窗口的三个指针我都设计成了单向增长的，即 base、nextToSend 和 rear 都是递增的。这样设计的好处是可以直接通过取模运算来获取下标，而不需要考虑指针的回退。

```

1 package com.ouc.tcp.test;
2
3 import com.ouc.tcp.client.Client;
4 import com.ouc.tcp.client.UDT_RetransTask;
5 import com.ouc.tcp.message.TCP_PACKET;
6
7 public class SenderWindow {

```

```
8     private int size;
9     private SenderElem[] window;
10    private int base;
11    private int nextToSend; // 下一个要发送的元素的下标
12    private int rear; // 窗口的最后一个元素的下标
13
14    public SenderWindow(int size) {
15        this.size = size;
16        this.window = new SenderElem[size];
17        for (int i = 0; i < size; i++) {
18            this.window[i] = new SenderElem();
19        }
20        this.base = 0;
21        this.nextToSend = 0;
22        this.rear = 0;
23    }
24
25    private int getIdx(int seq) { return seq % size; } // 取模运算获取下标
26
27    public boolean isFull() { return rear - base == size; }
28
29    public boolean isEmpty() { return base == rear; }
30
31    public boolean isAllSent() { return nextToSend == rear; }
32
33    public void pushPacket(TCP_PACKET packet) {
34        int idx = getIdx(rear);
35        // 初始化为未 ACK
36        window[idx].setElem(packet, SenderFlag.NOT_ACKED.ordinal());
37        rear++;
38    }
39
40    public void sendPacket(TCP_Sender sender, Client client, int delay, int
period) {
41        // 窗口为空或者窗口中的所有元素都已经发送
42        if (isEmpty() || isAllSent()) { return; }
43
44        int idx = getIdx(nextToSend);
45        TCP_PACKET pack = window[idx].getPacket();
46        window[idx].scheduleTask(new UDT_RetransTask(client, pack), delay,
period);
47        nextToSend++;
48
49        sender.udt_send(pack);
50    }
51
52    public void ackPacket(int seq) {
53        // 从 base 开始遍历, 找到对应的包并 ACK
54        for (int i = base; i != rear; i++) {
55            int idx = getIdx(i);
56            if (window[idx].getPacket().getTcpH().getTh_seq() == seq && !
window[idx].isAked()) {
57                window[idx].ackPacket();
58                break;
59            }
60        }
61
62        // 滑动窗口
```

```

63         while (base != rear && window[getIdx(base)].isAked()) {
64             int idx = getIdx(base);
65             window[idx].resetElem();
66             base++;
67         }
68     }
69
70 }
71

```

- TCP_Sender:

修改的地方依然用黄色高亮标出。和 RDT 3.0 相比，将计时器移入了窗口的元素中，每个数据包都有一个计时器。在将数据包加入窗口前判断窗口是否满，如果满了则等待窗口滑动。加入窗口时使用 clone 方法复制数据包，传入一个新的数据包对象。

```

1  enum WindowFlag {
2      FULL, NOT_FULL
3      // FULL: 窗口满
4      // NOT_FULL: 窗口未满
5  }
6
7  public class TCP_Sender extends TCP_Sender_ADT {
8
9      private TCP_PACKET tcpPack;    //待发送的 TCP 数据报
10     private volatile int flag = WindowFlag.NOT_FULL.ordinal(); // 窗口状态标志
11     private final SenderWindow window = new SenderWindow(16);
12
13     // ...
14
15     @Override
16     //可靠发送（应用层调用）：封装应用层数据，产生 TCP 数据报；需要修改
17     public void rdt_send(int dataIndex, int[] appData) {
18
19         //生成 TCP 数据报（设置序号和数据字段/校验和），注意打包的顺序
20         tcpH.setTh_seq(dataIndex * appData.length + 1); //包序号设置为字节流号：
21         tcpS.setData(appData);
22         tcpPack = new TCP_PACKET(tcpH, tcpS, destinAddr);
23         //更新带有 checksum 的 TCP 报文头
24         tcpH.setTh_sum(CheckSum.computeChkSum(tcpPack));
25         tcpPack.setTcpH(tcpH);
26
27         if (window.isFull()) {
28             //窗口满，等待窗口滑动
29             flag = WindowFlag.FULL.ordinal();
30         }
31         while (flag == WindowFlag.FULL.ordinal()) {
32             //等待窗口滑动
33         }
34
35         try {
36             window.pushPacket(tcpPack.clone()); // 将数据包加入窗口
37         } catch (CloneNotSupportedException e) {
38             throw new RuntimeException(e);
39         }
40
41         window.sendPacket(this, client, 1000, 1000); // 发送数据包
42     }

```

```

43
44 // ...
45
46 @Override
47 //需要修改
48 public void waitACK() {
49     //循环检查 ackQueue
50     //循环检查确认号对列中是否有新收到的 ACK
51     if (!ackQueue.isEmpty()) {
52         int currentAck = ackQueue.poll();
53         window.ackPacket(currentAck);
54         if (!window.isFull()) {
55             flag = WindowFlag.NOT_FULL.ordinal();
56         }
57     }
58 }
59
60 // ...
61
62 }

```

图 19 是接收方窗口的 UML 类图。接收方窗口的实现和发送方窗口的实现类似，只是接收方窗口不需要计时器，只需要维护一个窗口即可。实现上它只需要 1 个指针：**base**，指向窗口的左沿，是窗口下一个期望的包。之所以不需要 **rear** 是因为其序号有序，不需要额外的指针来标记。

每收到一个包，若校验码检查正确，则判断包的序号是否超出窗口右沿，如果在窗口内是则回复 ACK 并将包放入窗口，如果是重发包则回复 ACK 但不存入窗口，乱序包直接丢弃。若收到的包是窗口左沿的包，则交付数据并滑动窗口。校验码检查错误的包直接丢弃等待重发。

对每个 **ReceiverElem**，维护了一个数据包和一个包状态（等待包到来或已缓存）。而检查序号一步同样用了一个枚举类型表示包的状态，包括重发包、乱序（不在窗口内的包）和正常包，正常包又细分了一个窗口内的包和窗口左沿的包，这样可以更好地处理不同状态的包。

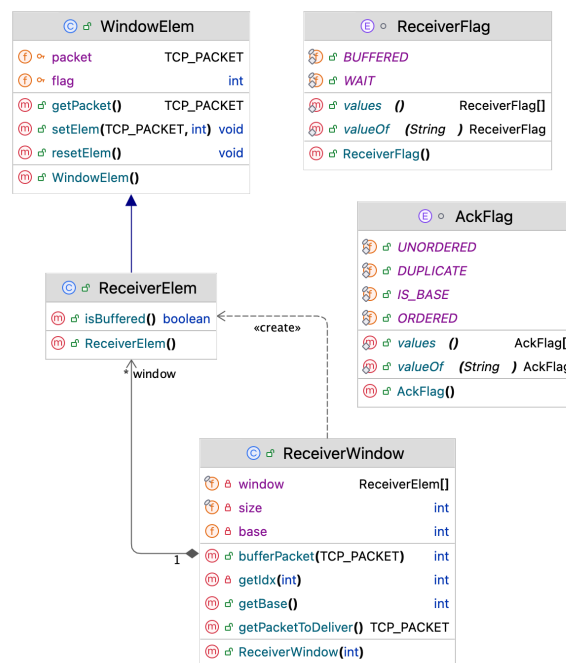


图 19 Receiver Window UML 类图

对于接收方窗口的实现，代码如下：

- ReceiverElem:

和 SenderElem 相比，ReceiverElem 几乎是直接继承自 WindowElem，只是增加了一个数据包的状态的布尔值。

```
1 package com.ouc.tcp.test;
2
3 enum ReceiverFlag {
4     WAIT, BUFFERED
5     // WAIT: 等待接收或已经确认
6     // BUFFERED: 已经接收但还未确认
7 }
8
9 public class ReceiverElem extends WindowElem {
10
11     public ReceiverElem() {
12         super();
13     }
14
15     public boolean isBuffered() {
16         return flag == ReceiverFlag.BUFFERED.ordinal();
17     }
18
19 }
```

- ReceiverWindow:

ReceiverWindow 只需要将包放入窗口和交付数据两个方法。对于包的状态使用 AckFlag 枚举类型表示。交付时，如果窗口左沿的包是已经缓存的包，则交付数据并滑动窗口。

```
1 package com.ouc.tcp.test;
2
3 import com.ouc.tcp.message.TCP_PACKET;
4
5 enum AckFlag {
6     ORDERED, DUPLICATE, UNORDERED, IS_BASE
7     // ORDERED: 接收到的包是按序的
8     // DUPLICATE: 接收到的包是重复的
9     // UNORDERED: 接收到的包提前送达，乱序
10    // IS_BASE: 接收到的包是基序号的包，开始交付数据
11 }
12
13 public class ReceiverWindow {
14     private final int size;
15     private final ReceiverElem[] window;
16     private int base; // 窗口的基序号，即下一个期望接收的包的序号
17
18
19     public ReceiverWindow(int size) {
20         this.size = size;
21         this.window = new ReceiverElem[size];
22         for (int i = 0; i < size; i++) {
23             this.window[i] = new ReceiverElem();
24         }
25         this.base = 0;
26     }
27
28     private int getIdx(int seq) { return seq % size; } // 取模运算获取下标
```



```

29
30     public int bufferPacket(TCP_PACKET packet) {
31         int seq = (packet.getTcpH().getTh_seq() - 1) /
packet.getTcpS().getData().length;
32
33         // 超前送达, 乱序
34         if (seq >= base + size) { return AckFlag.UNORDERED.ordinal(); }
35         // 在窗口左侧, 重复
36         if (seq < base) { return AckFlag.DUPLICATE.ordinal(); }
37         // 在窗口内, 缓存
38         window[getIdx(seq)].setElem(packet, ReceiverFlag.BUFFERED.ordinal());
39         // 在窗口左沿, 交付数据
40         if (seq == base) {
41             return AckFlag.IS_BASE.ordinal();
42         }
43         return AckFlag.ORDERED.ordinal();
44     }
45
46     public TCP_PACKET getPacketToDeliver() {
47         // 如果窗口左沿的包没有缓存, 则无法交付数据
48         if (!window[getIdx(base)].isBuffered()) {
49             return null;
50         }
51
52         // 交付数据
53         TCP_PACKET packet = window[getIdx(base)].getPacket();
54         window[getIdx(base)].resetElem();
55         base++;
56         return packet;
57     }
58 }

```

- TCP_Receiver:

修改的地方依然用黄色高亮标出。接收到数据包后, 先检查校验和, 然后调用 bufferPacket 方法将数据包放入窗口或响应。如果数据包是窗口左沿的包, 则交付数据。

```

1 public class TCP_Receiver extends TCP_Receiver_ADT {
2
3     private TCP_PACKET ackPack;    // 回复的 ACK 报文段
4     private ReceiverWindow window = new ReceiverWindow(16);
5
6     // ...
7
8     @Override
9     // 接收到数据报: 检查校验和, 设置回复的 ACK 报文段
10    public void rdt_rcv(TCP_PACKET rcvPack) {
11        int dataLength = rcvPack.getTcpS().getData().length;
12        // 检查校验码, 生成 ACK
13        if (Checksum.computeChkSum(rcvPack) == rcvPack.getTcpH().getTh_sum()) {
14            int bufferResult = window.bufferPacket(rcvPack);
15            if (bufferResult == AckFlag.ORDERED.ordinal()
16                || bufferResult == AckFlag.DUPLICATE.ordinal()
17                || bufferResult == AckFlag.IS_BASE.ordinal()) {
18                tcpH.setTh_ack(rcvPack.getTcpH().getTh_seq());
19                ackPack = new TCP_PACKET(tcpH, tcpS, rcvPack.getSourceAddr());
20                tcpH.setTh_sum(Checksum.computeChkSum(ackPack));
21                reply(ackPack);

```

```

22     }
23
24     if (bufferResult == AckFlag.IS_BASE.ordinal()) {
25         TCP_PACKET packet = window.getPacketToDeliver();
26         while (packet != null) {
27             dataQueue.add(packet.getTcpS().getData());
28             packet = window.getPacketToDeliver();
29         }
30     }
31
32     }
33     // 错误包不回复 ACK
34     System.out.println();
35
36     //交付数据
37     deliver_data();
38 }
39
40 // ...
41
42 }

```

继续保持 `tcpH.setTh_eflag((byte) 7)`，允许信道出错、丢包、延迟，运行 Select Response 代码。

327	2024-12-21 14:05:04:997 CST DATA_seq: 32101 LOSS NO_ACK	1340	2024-12-21 14:05:04:984 CST ACK_ack: 32001
328	2024-12-21 14:05:05:009 CST DATA_seq: 32201 ACKed	1341	2024-12-21 14:05:05:010 CST ACK_ack: 32201
329	2024-12-21 14:05:05:022 CST DATA_seq: 32301 ACKed	1342	2024-12-21 14:05:05:022 CST ACK_ack: 32301
330	2024-12-21 14:05:05:035 CST DATA_seq: 32401 ACKed	1343	2024-12-21 14:05:05:035 CST ACK_ack: 32401
331	2024-12-21 14:05:05:046 CST DATA_seq: 32501 ACKed	1344	2024-12-21 14:05:05:046 CST ACK_ack: 32501
332	2024-12-21 14:05:05:056 CST DATA_seq: 32601 ACKed	1345	2024-12-21 14:05:05:057 CST ACK_ack: 32601
333	2024-12-21 14:05:05:069 CST DATA_seq: 32701 ACKed	1346	2024-12-21 14:05:05:070 CST ACK_ack: 32701
334	2024-12-21 14:05:05:082 CST DATA_seq: 32801 ACKed	1347	2024-12-21 14:05:05:082 CST ACK_ack: 32801
335	2024-12-21 14:05:05:094 CST DATA_seq: 32901 ACKed	1348	2024-12-21 14:05:05:095 CST ACK_ack: 32901
336	2024-12-21 14:05:05:107 CST DATA_seq: 33001 ACKed	1349	2024-12-21 14:05:05:108 CST ACK_ack: 33001
337	2024-12-21 14:05:05:120 CST DATA_seq: 33101 ACKed	1350	2024-12-21 14:05:05:120 CST ACK_ack: 33101
338	2024-12-21 14:05:05:131 CST DATA_seq: 33201 DELAY NO_ACK	1351	2024-12-21 14:05:05:144 CST ACK_ack: 33301
339	2024-12-21 14:05:05:143 CST DATA_seq: 33301 ACKed	1352	2024-12-21 14:05:05:156 CST ACK_ack: 33401
340	2024-12-21 14:05:05:156 CST DATA_seq: 33401 ACKed	1353	2024-12-21 14:05:05:169 CST ACK_ack: 33501
341	2024-12-21 14:05:05:169 CST DATA_seq: 33501 ACKed	1354	2024-12-21 14:05:05:182 CST ACK_ack: 33601
342	2024-12-21 14:05:05:182 CST DATA_seq: 33601 ACKed	1355	2024-12-21 14:05:06:000 CST ACK_ack: 32101
343	2024-12-21 14:05:05:999 CST *Re: DATA_seq: 32101 ACKed		
344	2024-12-21 14:05:06:000 CST DATA_seq: 33701 ACKed		
345	2024-12-21 14:05:06:013 CST DATA_seq: 33801 ACKed		

图 20 Sender Log

图 21 Receiver Log

1205	2024-12-21 14:05:02:514 CST ACK_ack: 18601 DELAY	1205	2024-12-21 14:05:02:514 CST ACK_ack: 18601 DELAY
1206	2024-12-21 14:05:02:526 CST ACK_ack: 18701	1206	2024-12-21 14:05:02:526 CST ACK_ack: 18701
1207	2024-12-21 14:05:02:539 CST ACK_ack: 18801	1207	2024-12-21 14:05:02:539 CST ACK_ack: 18801
1208	2024-12-21 14:05:02:552 CST ACK_ack: 18901	1208	2024-12-21 14:05:02:552 CST ACK_ack: 18901
1209	2024-12-21 14:05:02:564 CST ACK_ack: 19001	1209	2024-12-21 14:05:02:564 CST ACK_ack: 19001
1210	2024-12-21 14:05:02:577 CST ACK_ack: 19101	1210	2024-12-21 14:05:02:577 CST ACK_ack: 19101
1211	2024-12-21 14:05:02:590 CST ACK_ack: 19201	1211	2024-12-21 14:05:02:590 CST ACK_ack: 19201
1212	2024-12-21 14:05:02:602 CST ACK_ack: 19301	1212	2024-12-21 14:05:02:602 CST ACK_ack: 19301
1213	2024-12-21 14:05:02:615 CST ACK_ack: 19401	1213	2024-12-21 14:05:02:615 CST ACK_ack: 19401
1214	2024-12-21 14:05:02:627 CST ACK_ack: 19501	1214	2024-12-21 14:05:02:627 CST ACK_ack: 19501
1215	2024-12-21 14:05:02:639 CST ACK_ack: 19601	1215	2024-12-21 14:05:02:639 CST ACK_ack: 19601
1216	2024-12-21 14:05:02:650 CST ACK_ack: 19701	1216	2024-12-21 14:05:02:650 CST ACK_ack: 19701
1217	2024-12-21 14:05:02:663 CST ACK_ack: 19801	1217	2024-12-21 14:05:02:663 CST ACK_ack: 19801
1218	2024-12-21 14:05:02:676 CST ACK_ack: 19901	1218	2024-12-21 14:05:02:676 CST ACK_ack: 19901
1219	2024-12-21 14:05:02:688 CST ACK_ack: 20001	1219	2024-12-21 14:05:02:688 CST ACK_ack: 20001
1220	2024-12-21 14:05:02:701 CST ACK_ack: 20101	1220	2024-12-21 14:05:02:701 CST ACK_ack: 20101
1221	2024-12-21 14:05:03:517 CST *Re: ACK_ack: 18601	1221	2024-12-21 14:05:03:517 CST *Re: ACK_ack: 18601
1222		1222	2024-12-21 14:05:03:517 CST ACK_ack: 20201

图 22 Sender Log

图 23 Receiver Log

不管是在 Sender Log 中还是在 Receiver Log 中，都可以很明显看出窗口的滑动。

比如图 20，从时间上看，32201 到 33601 之间的包都是一个接一个（时间间隔很短）发送的，说明它们在一个窗口中。而 32101 的包一开始丢失了，导致 Sender 迟迟等不到回复，而此时（发出 33601 后）窗口满，无法继续发送，所以在 32101 重传前出现了较大的时间间隔，直到 32101 重传后窗口滑动，Sender 继续发送数据。

而图 23 中则是由于 18601 的 ACK 延迟了，导致 Sender 迟迟无法收到 ACK，窗口满了无法继续发送，直到 18601 重传、Receiver 发送 ACK，Sender 才继续发送数据。

99997	1299653
99998	1299673
99999	1299689
100000	1299709
100001	

图 24 recvData

查看交付的数据包，可以看到接收方收到的数据包是有序的且数量为 100000，没有出现数据包的丢失或者重复。可以说明 Select Response 协议的实现是正确的。

综上，对于 Select Response 的关注点实现了以下几点：

- ☑ 检查 Log 文件，当出现错误（包含数据包出错、数据包或确认报丢包）时，是否发方在 3s 后超时重传对应数据包（**此处实现是 1s**）。常见错误：发送方重传的数据包并不是出现错误问题的数据包。
- ☑ 检查输出文件，不存在乱序的情况。

5. Go-Back-N

Go-Back-N 和 Select Response 相比，Sender 只在窗口左沿放置一个全局的计时器，而不是每个数据包都有一个计时器。当计时器超时时，重传窗口中的所有数据包。而 Receiver 也维护一个计时器，当计时器超时，发回当前最后一个连续收到的包的 ACK，即累积确认。

在实现上，第一个修改是将每个数据包的计时器移入了 SenderWindow 类中。原来的计时器的 RetransTask 是重发单个分组，现在需要重发窗口中的所有数据包，因此需要修改 RetransTask 的实现。在 SenderWindow 类中：

1. 加入子类 GBN_RetransTask 继承 UDT_RetransTask，重写 run 方法，实现重发整个窗口的数据包。要实现重发，需要将 TCP_Sender 传入，因为需要调用 udt_send 方法发送数据包。实现 sendWindow 方法，将窗口中的数据包逐个发送。

```

1 private UDT_Timer timer;
2 private int delay;
3 private int period;
4 private TCP_Sender sender;
5
6 public class GBN_RetransTask extends TimerTask {
7     private TCP_Sender sender;
8     private SenderWindow window;
9
10    public GBN_RetransTask(TCP_Sender sender, SenderWindow window) {
11        this.sender = sender;
12        this.window = window;
13    }
14
15    public void run() {
16        window.sendWindow();
17    }
18 }

```

```

19
20 public void resetTimer() {
21     timer.cancel();
22     timer = new UDT_Timer();
23     if (!isEmpty()) {
24         timer.schedule(new GBN_RetransTask(this), delay, period);
25     }
26 }
27
28 public void sendWindow() {
29     nextToSend = base;
30     while (nextToSend < rear) {
31         sendPacket();
32     }
33 }

```

2. 发包时依然逐个发送数据包，但是在发送第一个包时启动计时器。在收到 ACK 时，逐个确认包并重置计时器。

```

1 private final SenderWindow window;
2
3 /*构造函数*/
4 public TCP_Sender() {
5     super(); //调用超类构造函数
6     super.initTCP_Sender(this); //初始化 TCP 发送端
7     // 由于窗口调用了 this, 所以需要在构造函数中初始化
8     window = new SenderWindow(this, 16, 3000, 3000);
9
10 }
11
12 @Override
13 //可靠发送（应用层调用）：封装应用层数据，产生 TCP 数据报；需要修改
14 public void rdt_send(int dataIndex, int[] appData) {
15
16     //生成 TCP 数据报（设置序号和数据字段/校验和），注意打包的顺序
17     tcpH.setTh_seq(dataIndex * appData.length + 1); //包序号设置为字节流号：
18     tcpS.setData(appData);
19     tcpPack = new TCP_PACKET(tcpH, tcpS, destinAddr);
20     //更新带有 checksum 的 TCP 报文头
21     tcpH.setTh_sum(CheckSum.computeChkSum(tcpPack));
22     tcpPack.setTcpH(tcpH);
23
24     if (window.isFull()) {
25         //窗口满，等待窗口滑动
26         flag = WindowFlag.FULL.ordinal();
27     }
28     while (flag == WindowFlag.FULL.ordinal()) {
29         //等待窗口滑动
30     }
31
32     try {
33         window.pushPacket(tcpPack.clone());
34     } catch (CloneNotSupportedException e) {
35         throw new RuntimeException(e);
36     }
37
38     window.sendPacket();

```

```
39 }
```

- 加入发包、收 ACK 时计时器的逻辑。按照 PPT 上的说明，在左沿发送第一个包时启动计时器。由于在 Go-Back-N 中接收方发回的 ACK 是最后一个连续收到的包的 ACK，所以在收到 ACK 时可以让确认包的步骤和滑动窗口的步骤合并。每确认一个包（等同于滑动窗口），重置计时器。

```
1 private void sendPacket() {
2     if (isEmpty() || isAllSent()) {
3         // 窗口为空或者窗口中的所有元素都已经发送
4         return;
5     }
6
7     int idx = getIdx(nextToSend);
8     TCP_PACKET pack = window[idx].getPacket();
9
10    // 如果是第一个元素，启动定时器
11    if (atBase()) {
12        timer.schedule(new GBN_RetransTask(sender, this), delay, period);
13    }
14
15    nextToSend++;
16    sender.udt_send(pack);
17 }
18
19 public void ackPacket(int seq) {
20     int idx = getIdx(base);
21     for (int i = base; i != rear; i++) {
22         int idx = getIdx(i);
23         if (window[idx].getPacket().getTcpH().getTh_seq() <= ack && !
24             window[idx].isAked()) {
25             window[idx].ackPacket();
26             window[idx].resetElem();
27             base++;
28             resetTimer();
29         }
30     }
```

对于 Receiver，此时 Receiver 也需要维护一个计时器，当计时器超时，发回当前最后一个连续收到的包的 ACK，而基于 Select Response 的代码上，实际上这个 ACK 就是 $base - 1$ 的包的 ACK，因此和 Select Response 需要处理重复、窗口内或者窗口左沿相比，Go-Back-N 只需要处理窗口左沿的包。

在 TCP_Receiver 类中：

```
1 // 加入计时器
2 private UDT_Timer timer = new UDT_Timer();
3
4 // ...
5
6 @Override
7 // 接收到数据报：检查校验和，设置回复的 ACK 报文段
8 public void rdt_rcv(TCP_PACKET recvPack) {
9     // 等待 500ms，然后发回 ACK
10    timer.schedule(new TimerTask() {
11        @Override
```

```

12     public void run() {
13         reply(ackPack);
14         timer.cancel(); // 必须加入这一句, 否则收一个包就会开一个计时器
15         timer = new UDT_Timer();
16     }
17 }, 500); // delay: 500ms
18
19 //检查校验码, 生成 ACK
20 if (Checksum.computeChkSum(recvPack) == recvPack.getTcpH().getTh_sum()) {
21     int bufferResult = window.bufferPacket(recvPack);
22     // 如果是窗口左沿的包, 计时器计时 500ms 等待其余包到达
23     if (bufferResult == AckFlag.IS_BASE.ordinal()) {
24         TCP_PACKET packet = window.getPacketToDeliver();
25         while (packet != null) {
26             dataQueue.add(packet.getTcpS().getData());
27             tcpH.setTh_ack(packet.getTcpH().getTh_seq());
28             ackPack = new TCP_PACKET(tcpH, tcpS, recvPack.getSourceAddr());
29             tcpH.setTh_sum(Checksum.computeChkSum(ackPack));
30             packet = window.getPacketToDeliver();
31         }
32         // 如果有计时器在等待, 重置计时器, 只有在 500ms 期间没有收到基序号的包时才会发回 ACK
33         if (timer != null) {
34             timer.cancel();
35             timer = new UDT_Timer();
36             timer.schedule(
37                 new TimerTask() {
38                     public void run() {
39                         reply(ackPack); //回复 ACK 报文段
40                     }
41                 }, 500
42             );
43         }
44         // 如果是按序的包, 继续等待, 而如果是乱序或者重复的包则立即响应 ACK
45     } else if (bufferResult != AckFlag.ORDERED.ordinal()) {
46         reply(ackPack);
47     }
48 }
49 }
50
51 // ...
52
53 }

```

这一边计时器的设计比较巧妙。收包的情况可以分为三种：

1. 收到的包是窗口左沿的包，即 base 包，此时 Receiver 会交付数据并回复 ACK，然后计时器开始计时 500ms，等待其余包到达。如果 500ms 内没有收到基序号的包，则发回 ACK。
2. 收到的包是按序的包，Receiver 会继续等待，直到收到乱序或者重复的包，立即回复 ACK；或者收到窗口左沿的包，交付数据并继续等待。
3. 收到的包是乱序或者重复的包，Receiver 立即回复 ACK。

因此如果所有的包都按序依次到达，Receiver 最长会在窗口满后的 500ms 后回复 ACK。而如果有乱序或者重复的包，Receiver 会立即回复 ACK。因此需要要求 Sender 重发窗口的间隔要比两个 RTT 长，否则 Receiver 会在 500ms 后回复 ACK，而 Sender 会在 2s 后重发窗口，这样会导致 Receiver 重复回复 ACK。这里我按照 PPT 上的说明，给 Sender 设置了 3s 的延迟和重发间隔。

继续保持 `tcpH.setTh_eflag((byte) 7)`，允许信道出错、丢包、延迟，运行 Go-Back-N 代码。

55	2024-12-24 14:29:24:569 CST DATA_seq: 3601	NO_ACK
56	2024-12-24 14:29:24:582 CST DATA_seq: 3701	ACKed
57	2024-12-24 14:29:25:090 CST DATA_seq: 3801	NO_ACK
58	2024-12-24 14:29:25:103 CST DATA_seq: 3901	NO_ACK
59	2024-12-24 14:29:25:116 CST DATA_seq: 4001	NO_ACK
60	2024-12-24 14:29:25:129 CST DATA_seq: 4101	NO_ACK
61	2024-12-24 14:29:25:141 CST DATA_seq: 4201	NO_ACK
62	2024-12-24 14:29:25:154 CST DATA_seq: 4301	NO_ACK
63	2024-12-24 14:29:25:167 CST DATA_seq: 4401	NO_ACK
64	2024-12-24 14:29:25:180 CST DATA_seq: 4501	NO_ACK
65	2024-12-24 14:29:25:193 CST DATA_seq: 4601	NO_ACK
66	2024-12-24 14:29:25:206 CST DATA_seq: 4701	NO_ACK
67	2024-12-24 14:29:25:219 CST DATA_seq: 4801	NO_ACK
68	2024-12-24 14:29:25:232 CST DATA_seq: 4901	NO_ACK
69	2024-12-24 14:29:25:245 CST DATA_seq: 5001	NO_ACK
70	2024-12-24 14:29:25:258 CST DATA_seq: 5101	NO_ACK
71	2024-12-24 14:29:25:271 CST DATA_seq: 5201	NO_ACK
72	2024-12-24 14:29:25:284 CST DATA_seq: 5301	ACKed
73	2024-12-24 14:29:25:789 CST DATA_seq: 5401	NO_ACK
74	2024-12-24 14:29:25:801 CST DATA_seq: 5501	NO_ACK

图 25 Sender Log

1292	2024-12-24 14:29:25:088 CST ACK_ack: 3701
1293	2024-12-24 14:29:25:787 CST ACK_ack: 5301
1294	2024-12-24 14:29:26:480 CST ACK_ack: 6901
1295	2024-12-24 14:29:27:025 CST ACK_ack: 7301
1296	2024-12-24 14:29:30:032 CST ACK_ack: 8901

图 26 Receiver Log

在 Go-Back-N 中，如果发送无异常，在 Sender Log 中会看到只有窗口的最后一个包被确认（因为收方计时器设置得远比传输时间长）。收方的计时器间隔为 500ms，因此大约每 500ms 发送一个 ACK。

1323	2024-12-24 14:29:38:907 CST ACK_ack: 28301
1324	2024-12-24 14:29:41:915 CST ACK_ack: 29901
1325	💡 2024-12-24 14:29:41:915 CST ACK_ack: 29901
1326	2024-12-24 14:29:41:915 CST ACK_ack: 29901
1327	2024-12-24 14:29:41:916 CST ACK_ack: 29901
1328	2024-12-24 14:29:41:916 CST ACK_ack: 29901
1329	2024-12-24 14:29:41:916 CST ACK_ack: 29901
1330	2024-12-24 14:29:41:916 CST ACK_ack: 29901
1331	2024-12-24 14:29:41:916 CST ACK_ack: 29901
1332	2024-12-24 14:29:41:916 CST ACK_ack: 29901
1333	2024-12-24 14:29:41:917 CST ACK_ack: 29901
1334	2024-12-24 14:29:41:917 CST ACK_ack: 29901
1335	2024-12-24 14:29:41:917 CST ACK_ack: 29901
1336	2024-12-24 14:29:41:917 CST ACK_ack: 29901
1337	2024-12-24 14:29:41:917 CST ACK_ack: 29901
1338	2024-12-24 14:29:41:917 CST ACK_ack: 29901
1339	2024-12-24 14:29:42:609 CST ACK_ack: 31501
1340	2024-12-24 14:29:43:306 CST ACK_ack: 33101

图 27 Receiver Log

319	2024-12-24 14:29:38:414 CST DATA_seq: 28401 WRONG NO_ACK
320	2024-12-24 14:29:38:427 CST DATA_seq: 28501 NO_ACK
321	2024-12-24 14:29:38:439 CST DATA_seq: 28601 NO_ACK
322	2024-12-24 14:29:38:452 CST DATA_seq: 28701 NO_ACK
323	2024-12-24 14:29:38:467 CST DATA_seq: 28801 NO_ACK
324	2024-12-24 14:29:38:480 CST DATA_seq: 28901 NO_ACK
325	2024-12-24 14:29:38:493 CST DATA_seq: 29001 NO_ACK
326	2024-12-24 14:29:38:505 CST DATA_seq: 29101 NO_ACK
327	2024-12-24 14:29:38:518 CST DATA_seq: 29201 NO_ACK
328	2024-12-24 14:29:38:531 CST DATA_seq: 29301 NO_ACK
329	2024-12-24 14:29:38:543 CST DATA_seq: 29401 NO_ACK
330	2024-12-24 14:29:38:556 CST DATA_seq: 29501 NO_ACK
331	2024-12-24 14:29:38:569 CST DATA_seq: 29601 NO_ACK
332	2024-12-24 14:29:38:582 CST DATA_seq: 29701 NO_ACK
333	2024-12-24 14:29:38:907 CST DATA_seq: 29801 NO_ACK
334	2024-12-24 14:29:38:920 CST DATA_seq: 29901 ACKed
335	2024-12-24 14:29:41:909 CST *Re: DATA_seq: 28401 NO_ACK
336	2024-12-24 14:29:41:909 CST *Re: DATA_seq: 28501 NO_ACK
337	2024-12-24 14:29:41:909 CST *Re: DATA_seq: 28601 NO_ACK
338	2024-12-24 14:29:41:909 CST *Re: DATA_seq: 28701 NO_ACK
339	2024-12-24 14:29:41:909 CST *Re: DATA_seq: 28801 NO_ACK
340	2024-12-24 14:29:41:909 CST *Re: DATA_seq: 28901 NO_ACK
341	2024-12-24 14:29:41:909 CST *Re: DATA_seq: 29001 NO_ACK
342	2024-12-24 14:29:41:909 CST *Re: DATA_seq: 29101 NO_ACK
343	2024-12-24 14:29:41:909 CST *Re: DATA_seq: 29201 NO_ACK
344	2024-12-24 14:29:41:909 CST *Re: DATA_seq: 29301 NO_ACK
345	2024-12-24 14:29:41:909 CST *Re: DATA_seq: 29401 NO_ACK
346	2024-12-24 14:29:41:909 CST *Re: DATA_seq: 29501 NO_ACK
347	2024-12-24 14:29:41:909 CST *Re: DATA_seq: 29601 NO_ACK
348	2024-12-24 14:29:41:909 CST *Re: DATA_seq: 29701 NO_ACK
349	2024-12-24 14:29:41:909 CST *Re: DATA_seq: 29801 NO_ACK
350	2024-12-24 14:29:41:909 CST *Re: DATA_seq: 29901 ACKed

图 28 Sender Log

而一旦分组出现问题或者 ACK 丢失、延迟，就会出现整个分组的重传。比如如图所示就是 28401 的错误导致了整个窗口的超时重传。

99992	1299583
99993	1299601
99994	1299631
99995	1299637
99996	1299647
99997	1299653
99998	1299673
99999	1299689
100000	1299709

图 29 recvData

依然查看交付的数据包，可以看到也没有问题。

综上，对于 Go-Back-N 的关注点实现了以下几点：

- ☒ 检查 Log 文件，当出现错误（包含数据包出错、数据包或确认报丢包）时，发方在 3s 后超时重传当前窗口内的所有包；超时重传的包包含发送端已经收到确认的包。
- ☒ 检查接收方有没有做为期 500ms 的累计确认。
- ☒ 检查输出文件，是否对应包的正确内容包含在输出文件中，而不是包含出错的包内容以及重复的包内容。
- ☐ 接收方使用了窗口缓存，按 GB 协议，接收方不缓存失序的包，但我的代码基于 Select Response 的代码，考虑到 TCP 要求缓存失序包，我没有修改这一点。

6. TCP

不需要拥塞控制的 TCP 相当于 Select Response 的 Receiver 和 Go-Back-N 的 Sender。

TCP ACK 产生 [RFC 1122, RFC 2581]

<i>event at receiver</i>	<i>TCP receiver action</i>
收到的数据包满足expected seq #. expected seq # 之前的数据包已经 ACKed	延迟发送 ACK. 等待500ms 看看有没有下一个包. 没有新包到达, 发送ACK
收到的数据包满足expected seq #. 之前的数据包的ACK还未发送	等待累计确认计时器到期发送ACK; 进行累积确认
收到数据包的Seq#比expected seq# 大; 检测出Gap	立即发送 <i>duplicate ACK</i> , 指出当前的expected seq#
收到数据包的seq#填入到接收缓存 的Gap里	累积确认发送ACK, 提供当前Gap左 边的seq#

按照 PPT 上的说明, 如果接收方收到的数据包恰为期望的数据包, 则延迟 500ms 发回 ACK, 而如果收到的数据包不是期望的数据包, 则立即发回 ACK。这一点在 Go-Back-N 中已经实现, 因此只需沿用对应的代码即可。

为了后面迭代实现 TCP 的拥塞控制做准备, 我将 SenderWindow 的数据类型改为 LinkedBlockingDeque, 来自 java.util.concurrent 包。这样可以更方便地实现慢开始和拥塞避免。

因此, 修改 SenderWindow 类如下:

- 对于发送包计时器的实现, 和 Go-Back-N 类似, 只是判断条件由比较 base 改为判断窗口是否为空。

```

1 public boolean isEmpty() {
2     return window.isEmpty();
3 }
4
5 public void pushPacket(TCP_PACKET packet) {
6     // 如果窗口为空, 启动定时器
7     if (isEmpty()) {
8         timer = new UDT_Timer();
9         timer.schedule(new GBN_RetransTask(this), delay, period);
10    }
11    window.push(new SenderElem(packet, SenderFlag.NOT_ACKED.ordinal()));
12 }
```

- 发送窗口时, 逐个发送窗口中的数据包。而发送包的时候, 滚动队列, 将窗口左沿的包取出并发送, 然后将其放回队列。

```

1 public void sendWindow() {
2     // 发送窗口中的数据
3     for (SenderElem elem : window) {
4         if (!elem.isAked()) {
5             sender.udt_send(elem.getPacket());
6         }
7     }
8 }
9
```

```

10 public void sendPacket() {
11     SenderElem elem = window.poll();
12     if (elem == null) {
13         return;
14     }
15     if (!elem.isAcked()) {
16         sender.udt_send(elem.getPacket());
17     }
18     window.push(elem);
19 }

```

- 响应 ACK 时，逐个确认包并重置计时器。这里可以直接调用 remove 方法清除已经确认的包。

```

1 public void ackPacket(int ack) {
2     for (SenderElem elem : window) {
3         if (elem.getPacket().getTcpH().getTh_seq() <= ack) {
4             elem.ackPacket();
5             window.remove(elem);
6             resetTimer();
7         }
8     }
9 }

```

而由于接收方的代码沿用自 Select Response 修改成的 Go-Back-N，有乱序包缓存和延迟 500ms 累积确认的能力，因此不需要修改。

依然是保持 tcpH.setTh_eflag((byte) 7)，允许信道出错、丢包、延迟，运行 TCP 代码。

3	2024-12-24 14:24:52:323 CST DATA_seq: 1	NO_ACK
4	2024-12-24 14:24:52:338 CST DATA_seq: 101	NO_ACK
5	2024-12-24 14:24:52:354 CST DATA_seq: 201	NO_ACK
6	2024-12-24 14:24:52:369 CST DATA_seq: 301	NO_ACK
7	2024-12-24 14:24:52:383 CST DATA_seq: 401	NO_ACK
8	2024-12-24 14:24:52:400 CST DATA_seq: 501	NO_ACK
9	2024-12-24 14:24:52:415 CST DATA_seq: 601	NO_ACK
10	2024-12-24 14:24:52:428 CST DATA_seq: 701	NO_ACK
11	2024-12-24 14:24:52:441 CST DATA_seq: 801	NO_ACK
12	2024-12-24 14:24:52:454 CST DATA_seq: 901	NO_ACK
13	2024-12-24 14:24:52:468 CST DATA_seq: 1001	NO_ACK
14	2024-12-24 14:24:52:481 CST DATA_seq: 1101	NO_ACK
15	2024-12-24 14:24:52:493 CST DATA_seq: 1201	NO_ACK
16	2024-12-24 14:24:52:506 CST DATA_seq: 1301	NO_ACK
17	2024-12-24 14:24:52:519 CST DATA_seq: 1401	NO_ACK
18	2024-12-24 14:24:52:532 CST DATA_seq: 1501	ACKed

图 31 Sender Log

1181	2024-12-24 14:24:53:034 CST ACK_ack: 1501
1182	2024-12-24 14:24:53:641 CST ACK_ack: 2401
1183	2024-12-24 14:24:57:157 CST ACK_ack: 4001
1184	2024-12-24 14:24:57:843 CST ACK_ack: 5501
1185	2024-12-24 14:25:01:357 CST ACK_ack: 7101
1186	2024-12-24 14:25:02:050 CST ACK_ack: 8701

图 32 Receiver Log

和 Go-Back-N 类似，累积确认的 ACK 会在 500ms 后发出，正常只确认窗口最右侧的包。

1047	2024-12-24 14:26:00:770 CST DATA_seq: 88401 LOSS NO_ACK
1048	2024-12-24 14:26:00:783 CST DATA_seq: 88501 NO_ACK
1049	2024-12-24 14:26:00:795 CST DATA_seq: 88601 NO_ACK
1050	2024-12-24 14:26:00:807 CST DATA_seq: 88701 NO_ACK
1051	2024-12-24 14:26:00:819 CST DATA_seq: 88801 NO_ACK
1052	2024-12-24 14:26:00:832 CST DATA_seq: 88901 NO_ACK
1053	2024-12-24 14:26:00:844 CST DATA_seq: 89001 NO_ACK
1054	2024-12-24 14:26:00:857 CST DATA_seq: 89101 NO_ACK
1055	2024-12-24 14:26:00:870 CST DATA_seq: 89201 NO_ACK
1056	2024-12-24 14:26:00:882 CST DATA_seq: 89301 NO_ACK
1057	2024-12-24 14:26:00:898 CST DATA_seq: 89401 NO_ACK
1058	2024-12-24 14:26:01:261 CST DATA_seq: 89501 NO_ACK
1059	2024-12-24 14:26:01:273 CST DATA_seq: 89601 NO_ACK
1060	2024-12-24 14:26:01:286 CST DATA_seq: 89701 NO_ACK
1061	2024-12-24 14:26:01:298 CST DATA_seq: 89801 NO_ACK
1062	2024-12-24 14:26:01:311 CST DATA_seq: 89901 NO_ACK
1063	2024-12-24 14:26:04:266 CST *Re: DATA_seq: 89901 ACKed
1064	2024-12-24 14:26:04:266 CST *Re: DATA_seq: 89801 NO_ACK
1065	2024-12-24 14:26:04:266 CST *Re: DATA_seq: 89701 NO_ACK
1066	2024-12-24 14:26:04:266 CST *Re: DATA_seq: 89601 NO_ACK
1067	2024-12-24 14:26:04:266 CST *Re: DATA_seq: 89501 NO_ACK
1068	2024-12-24 14:26:04:266 CST *Re: DATA_seq: 89401 NO_ACK
1069	2024-12-24 14:26:04:266 CST *Re: DATA_seq: 89301 NO_ACK
1070	2024-12-24 14:26:04:266 CST *Re: DATA_seq: 89201 NO_ACK
1071	2024-12-24 14:26:04:266 CST *Re: DATA_seq: 89101 NO_ACK
1072	2024-12-24 14:26:04:266 CST *Re: DATA_seq: 89001 NO_ACK
1073	2024-12-24 14:26:04:266 CST *Re: DATA_seq: 88901 NO_ACK
1074	2024-12-24 14:26:04:266 CST *Re: DATA_seq: 88801 NO_ACK
1075	2024-12-24 14:26:04:266 CST *Re: DATA_seq: 88701 NO_ACK
1076	2024-12-24 14:26:04:266 CST *Re: DATA_seq: 88601 NO_ACK
1077	2024-12-24 14:26:04:266 CST *Re: DATA_seq: 88501 NO_ACK
1078	2024-12-24 14:26:04:266 CST *Re: DATA_seq: 88401 NO_ACK

图 33 Sender Log

1242	2024-12-24 14:26:00:707 CST ACK_ack: 87801
1243	2024-12-24 14:26:01:260 CST ACK_ack: 88301
1244	2024-12-24 14:26:04:773 CST ACK_ack: 89901
1245	2024-12-24 14:26:05:501 CST ACK_ack: 91501

图 34 Receiver Log

对于丢包导致的重传，延迟 3s 之后整个窗口进行重传。

99992	1299583
99993	1299601
99994	1299631
99995	1299637
99996	1299647
99997	1299653
99998	1299673
99999	1299689
100000	1299709

图 35 recvData

查看交付的数据包，可以看到没有什么问题。

综上，对于 TCP 的关注点实现了以下几点：

- 检查 Log 文件，当出现错误（包含数据包出错、数据包或确认报丢包）时，发方在 3s 后超时重传当前窗口内的所有未收到确认的包；超时重传的包包含发送端已经收到确认的包。
- 检查接收方做了为期 500ms 的累计确认。
- 检查输出文件，对应包的正确内容包含在输出文件中，而不是包含出错的包内容以及重复的包内容。

- ✓ 发送端使用 `LinkedBlockingDeque` 作为窗口数据结构，方便实现慢开始和拥塞避免。窗口随着 ACK 的到来右移，窗口空间用完后以 `IS_FULL` 的标志阻止应用层继续调用。
- ✓ 发送端使用一个计时器来完成对于当前窗口内的所有未收到确认的包进行超时重传。
- ✓ 接收端使用了一个计时器来完成为期 500ms 的累计确认。
- ✓ 接收端使用了窗口缓存失序的包，累积确认后提交连续的报文段。

接下来加入拥塞控制，实现 TCP 的完整功能。

7. TCP Tahoe

TCP Tahoe 用 3 个方法来拥塞控制，分别是慢开始、拥塞避免和快重传。

1. **慢开始**由 `cwnd` 和 `ssthresh` 两个变量控制。`cwnd` 表示当前窗口大小，`ssthresh` 表示慢开始阈值。在慢开始阶段，`cwnd` 每收到一个 ACK 就加 1，直到 `cwnd` 达到 `ssthresh`，相当于每次翻倍（指数增大）然后进入拥塞避免阶段。
2. **拥塞避免**阶段，`cwnd` 按照 $1/cwnd$ 的比例增加，总体发包的数量就变成了 1，进入加法增大阶段
3. **快重传**是指当连续收到 3 个相同的 ACK 时，`ssthresh` 减半，`cwnd` 重置为 1，并重传窗口左沿的包。

SenderWindow 类修改如下：

1. 加入 `cwnd`、`dCwnd` 和 `ssthresh` 三个变量，分别表示当前窗口大小、拥塞窗口大小和慢开始阈值。

```
1 private int cwnd = 1; // 等同于原来的窗口大小 size
2 private double dCwnd = 1.0; // 浮点类型的 cwnd，用于拥塞避免
3 private int ssthresh = 16; // 慢开始阈值
```

2. 修改 `ackPacket` 方法，实现慢开始和拥塞避免。

在慢开始阶段，每收到一个 ACK，`cwnd` 加 1，实现乘法增大；在拥塞避免阶段，`cwnd` 按照 $1/cwnd$ 的比例增加，总体发包的数量就变成了 1。也就是这一步计算需要用到浮点值的 `cwnd`。

```
1 private void resendPacket(int ack) {
2     // 重传窗口左沿的包，100 是包的长度，用于计算字节流号
3     int expectedSeq = ack + 100;
4     for (SenderElem elem : window) {
5         if (elem.getPacket().getTcph().getTh_seq() == expectedSeq) {
6             sender.udt_send(elem.getPacket());
7             break;
8         }
9     }
10 }
11
12 public void ackPacket(int ack) {
13     // 滑动窗口
14     for (SenderElem elem : window) {
```

```

15         if (elem.getPacket().getTcpH().getTh_seq() <= ack) { // 确认序号小于等于
ack 的包
16             elem.setAked();
17             window.remove(elem);
18             if (cwnd < ssthresh) {
19                 // 指数增大, 慢开始
20                 cwnd++;
21                 dCwnd = cwnd;
22             }
23             // 每次收到 ACK 都重置计时器
24             resetTimer();
25         }
26     }
27
28     if (cwnd >= ssthresh) {
29         // 拥塞避免, 加法增大
30         dCwnd += (double) 1 / cwnd;
31         cwnd = (int) dCwnd;
32     }
33
34     // 快重传, 连续收到 3 个相同的 ACK 则重传窗口左沿的包
35     if (ack == lastAck) {
36         lastAckCount++;
37     } else {
38         lastAck = ack;
39         lastAckCount = 1;
40     }
41     if (lastAckCount >= lastAckCountLimit) {
42         // 拥塞避免, 重置窗口大小
43         ssthresh = cwnd / 2;
44         cwnd = 1;
45         dCwnd = (double) cwnd;
46         // 快重传
47         resendPacket(ack);
48     }
49 }

```

对于 TCP_Receiver 也需要做一些修改, 主要原因是在快重传和超时重传的“冲突”。超时重传一次传整个窗口的包, 会导致 Receiver 响应 ACK 的时候, Sender 会收到多个 ACK, 这样就会导致快重传的条件被满足。按照要求, 应当对于所有的按序到达的包或者已经被确认过的包 (重发的包) 都延迟 500ms 累积确认, 而不只是窗口左沿的包。

在 TCP_Receiver 类中:

```

1 public void rdt_recv(TCP_PACKET recvPack) {
2     // ...
3     if (Checksum.computeChkSum(recvPack) == recvPack.getTcpH().getTh_sum()) {
4         int bufferResult = window.bufferPacket(recvPack);
5         if (bufferResult == AckFlag.ORDERED.ordinal() || bufferResult ==
AckFlag.DUPLICATE.ordinal()) {
6             // ...
7         } else {
8             reply(ackPack);
9         }
10    }
11    // ...
12 }

```

在 ReceiverWindow 类中，IS_BASE 则不再需要，因为 Receiver 不再需要对窗口左沿的包进行特殊处理。

```

1 public int bufferPacket(TCP_PACKET packet) {
2     int seq = (packet.getTcpH().getTh_seq() - 1) / packet.getTcpS().getData().length;
3     if (seq >= base + size) {
4         return AckFlag.UNORDERED.ordinal();
5     }
6     int idx = getIdx(seq);
7     if (seq < base) {
8         return AckFlag.DUPLICATE.ordinal();
9     }
10    window[idx].setElem(packet, ReceiverFlag.BUFFERED.ordinal());
11    //     if (seq == base) {
12    //         return AckFlag.IS_BASE.ordinal();
13    //     }
14    return AckFlag.ORDERED.ordinal();
15 }

```

在运行过程中记录下 cwnd 和 ssthresh 的变化，使用 Python 绘制图表（截取了前 40 s）：

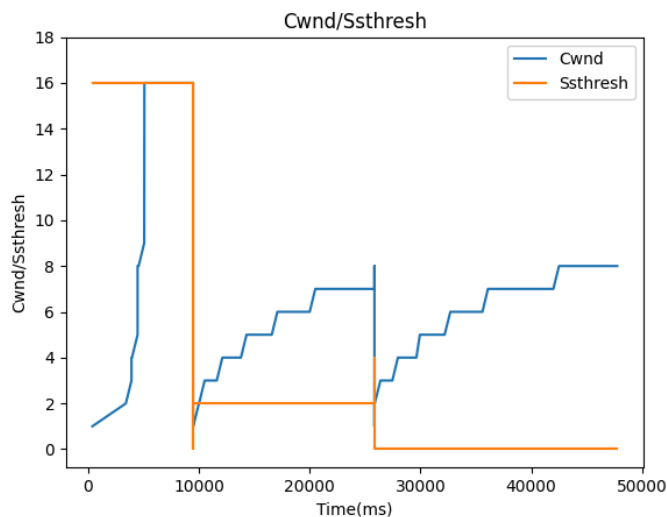


图 36 TCP Tahoe 中 cwnd 和 ssthresh 的变化

可以很明显看出慢开始、拥塞避免的过程。在慢开始阶段，cwnd 指数增长，而在拥塞避免阶段，cwnd 线性增长。快重传后，ssthresh 减半，cwnd 重置为 1，重新开始慢开始。

也可以在 Log 中体现出慢开始的过程：

3	2024-12-24 21:57:43:545 CST DATA_seq: 1	ACKed
4	2024-12-24 21:57:44:056 CST DATA_seq: 101	NO_ACK
5	2024-12-24 21:57:44:071 CST DATA_seq: 201	ACKed
6	2024-12-24 21:57:44:574 CST DATA_seq: 301	NO_ACK
7	2024-12-24 21:57:44:588 CST DATA_seq: 401	NO_ACK
8	2024-12-24 21:57:44:602 CST DATA_seq: 501	NO_ACK
9	2024-12-24 21:57:44:616 CST DATA_seq: 601	ACKed
10	2024-12-24 21:57:45:124 CST DATA_seq: 701	NO_ACK
11	2024-12-24 21:57:45:137 CST DATA_seq: 801	NO_ACK
12	2024-12-24 21:57:45:150 CST DATA_seq: 901	NO_ACK
13	2024-12-24 21:57:45:163 CST DATA_seq: 1001	NO_ACK
14	2024-12-24 21:57:45:176 CST DATA_seq: 1101	NO_ACK
15	2024-12-24 21:57:45:190 CST DATA_seq: 1201	NO_ACK
16	2024-12-24 21:57:45:203 CST DATA_seq: 1301	NO_ACK
17	2024-12-24 21:57:45:216 CST DATA_seq: 1401	ACKed
18	2024-12-24 21:57:45:723 CST DATA_seq: 1501	NO_ACK
19	2024-12-24 21:57:45:736 CST DATA_seq: 1601	NO_ACK

图 37 Sender Log

可以看到第一个包被确认后，接下来开始计数的第二个包也被确认，然后是接下来的第四个、第八个包，这是慢开始的过程。

852	2024-12-24 22:43:47:082 CST DATA_seq: 77501 LOSS NO_ACK
853	2024-12-24 22:43:47:095 CST DATA_seq: 77601 NO_ACK
854	2024-12-24 22:43:47:107 CST DATA_seq: 77701 NO_ACK
855	2024-12-24 22:43:47:120 CST DATA_seq: 77801 NO_ACK
856	2024-12-24 22:43:47:133 CST DATA_seq: 77901 NO_ACK
857	2024-12-24 22:43:47:145 CST DATA_seq: 78001 NO_ACK
858	2024-12-24 22:43:47:158 CST DATA_seq: 78101 NO_ACK
859	2024-12-24 22:43:47:171 CST DATA_seq: 78201 NO_ACK
860	2024-12-24 22:43:47:184 CST DATA_seq: 78301 NO_ACK
861	2024-12-24 22:43:47:196 CST DATA_seq: 78401 NO_ACK
862	2024-12-24 22:43:47:209 CST DATA_seq: 78501 NO_ACK
863	2024-12-24 22:43:47:222 CST DATA_seq: 78601 ACKed
864	2024-12-24 22:43:47:971 CST *Re: DATA_seq: 77501 NO_ACK

图 38 Sender Log

1162	2024-12-24 22:43:45:792 CST ACK_ack: 75001
1163	2024-12-24 22:43:46:438 CST ACK_ack: 76201
1164	2024-12-24 22:43:47:081 CST ACK_ack: 77401
1165	2024-12-24 22:43:47:727 CST ACK_ack: 77401
1166	2024-12-24 22:43:47:971 CST ACK_ack: 77401
1167	2024-12-24 22:43:48:476 CST ACK_ack: 78601
1168	2024-12-24 22:43:49:044 CST ACK_ack: 79201

图 39 Receiver Log

快重传的过程则是连续收到 3 个相同的 ACK，然后重传窗口左沿的包。这里 Sender 的 77501 包丢失，导致了 Receiver 无法收到期望的包，然后连续发了 3 个 99601 包的 ACK，Sender 立即就重传了 99701 包。

896	2024-12-24 22:46:38:077 CST ACK_ack: 43101 LOSS	497	2024-12-24 22:46:37:572 CST DATA_seq: 43101 ACKed
897	2024-12-24 22:46:40:514 CST *Re: ACK_ack: 43101	498	2024-12-24 22:46:40:513 CST *Re: DATA_seq: 43101 ACKed
898	2024-12-24 22:46:40:514 CST ACK_ack: 43101	499	2024-12-24 22:46:40:513 CST *Re: DATA_seq: 43001 NO_ACK
899	2024-12-24 22:46:40:514 CST ACK_ack: 43101	500	2024-12-24 22:46:40:513 CST *Re: DATA_seq: 42901 NO_ACK
900	2024-12-24 22:46:40:514 CST ACK_ack: 43101	501	2024-12-24 22:46:40:513 CST *Re: DATA_seq: 42801 NO_ACK
901	2024-12-24 22:46:40:514 CST ACK_ack: 43101	502	2024-12-24 22:46:40:513 CST *Re: DATA_seq: 42701 NO_ACK
902	2024-12-24 22:46:40:514 CST ACK_ack: 43101	503	2024-12-24 22:46:40:513 CST *Re: DATA_seq: 42601 NO_ACK

图 41 Sender Log

图 40 Receiver Log

再比如 43101 的 ACK 丢失，导致 Sender 超时重传了整个窗口。

所以对于 Tahoe 的关注点实现了以下几点：

- ✓ 检查 Log 文件，当出现错误（包含数据包出错、数据包或确认报丢包）时，发方在 3s 后超时重传当前窗口内的所有未收到确认的包；超时重传的包包含发送端已经收到确认的包，基本 TCP 要点实现正确。
- ✓ 使用 `LinkedBlockingDeque` 作为窗口数据结构，方便实现慢开始和拥塞避免。
- ✓ 出现超时重传后，窗口大小减小使用 `cwnd` 和 `ssthresh` 两个变量控制，慢开始、拥塞避免和快重传的过程符合 Tahoe 的要求。变量的改变影响了 `isFull` 方法的判断，而已经在窗口内的包由于仍可能需要重传，因此不会立即删除，只有在确认后才会删除。

8. TCP Reno

Reno 则是在 Tahoe 的基础上加入了快恢复机制，即当连续收到 3 个相同的 ACK 时，`ssthresh` 减半，`cwnd` 重置为 `ssthresh` 而不是 1。查了一下了解到也有不少人（比如 Linux 内核中）设计成 `cwnd` 重置为 `ssthresh + 3`（3 是 `lastAckCountLimit`），这样相当于对原来进行了一个修正。因此只需修改：

```
1 public void ackPacket(int ack) {
2
3     // ...
4
5     if (lastAckCount >= lastAckCountLimit) {
6         // 拥塞避免, 重置窗口大小
7         ssthresh = cwnd / 2;
8         cwnd = ssthresh;
9         dCwnd = (double) cwnd;
10        // 快重传
11        resendPacket(ack);
12    }
13 }
```

依然记录下 cwnd 和 ssthresh 的变化, 使用 Python 绘制图表 (截取了前 10000 ms):

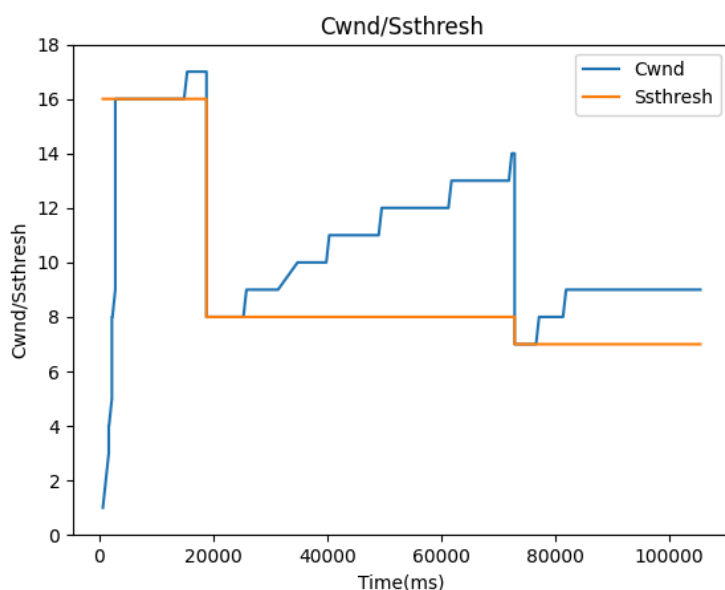


图 42 cwnd 和 ssthresh 的变化

可以看到 Reno 的 cwnd 变化和 Tahoe 有所不同, 快恢复机制使得 cwnd 在快恢复阶段线性增长, 而不是慢开始阶段的指数增长。在平均情况下 Reno 的传输效率要高于 Tahoe。

对于前面迭代的版本所验证的出错、丢包、延迟的问题, TCP Reno 也能很好地解决。

302	2024-12-24 23:18:28:358 CST DATA_seq: 28301	ACKed		
303	2024-12-24 23:18:28:370 CST DATA_seq: 28401	WRONG NO_ACK	1108	2024-12-24 23:18:27:570 CST ACK_ack: 25901
304	2024-12-24 23:18:28:386 CST DATA_seq: 28501	NO_ACK	1109	2024-12-24 23:18:28:279 CST ACK_ack: 27601
305	2024-12-24 23:18:28:398 CST DATA_seq: 28601	NO_ACK	1110	2024-12-24 23:18:28:992 CST ACK_ack: 28301
306	2024-12-24 23:18:28:411 CST DATA_seq: 28701	NO_ACK	1111	2024-12-24 23:18:29:069 CST ACK_ack: 28301
307	2024-12-24 23:18:28:424 CST DATA_seq: 28801	NO_ACK	1112	2024-12-24 23:18:29:562 CST ACK_ack: 28301
308	2024-12-24 23:18:28:437 CST DATA_seq: 28901	NO_ACK	1113	2024-12-24 23:18:30:068 CST ACK_ack: 29901
309	2024-12-24 23:18:28:450 CST DATA_seq: 29001	NO_ACK	1114	2024-12-24 23:18:30:650 CST ACK_ack: 29901
310	2024-12-24 23:18:28:463 CST DATA_seq: 29101	NO_ACK		
311	2024-12-24 23:18:28:476 CST DATA_seq: 29201	NO_ACK		
312	2024-12-24 23:18:28:488 CST DATA_seq: 29301	NO_ACK		
313	2024-12-24 23:18:28:992 CST DATA_seq: 29401	NO_ACK		
314	2024-12-24 23:18:29:005 CST DATA_seq: 29501	NO_ACK		
315	2024-12-24 23:18:29:018 CST DATA_seq: 29601	NO_ACK		
316	2024-12-24 23:18:29:031 CST DATA_seq: 29701	NO_ACK		
317	2024-12-24 23:18:29:043 CST DATA_seq: 29801	NO_ACK		
318	2024-12-24 23:18:29:056 CST DATA_seq: 29901	ACKed		
319	2024-12-24 23:18:29:069 CST DATA_seq: 30001	NO_ACK		
320	2024-12-24 23:18:29:562 CST *Re: DATA_seq: 28401	NO_ACK		

图 43 Sender Log

图 44 Receiver Log

比如这里的 78601 丢失对应数据包的快重传处理。

465	2024-12-24 23:18:42:113 CST DATA_seq: 43701 NO_ACK	1034	2024-12-24 23:18:42:010 CST ACK_ack: 42801
466	2024-12-24 23:18:45:016 CST *Re: DATA_seq: 43701 ACKed	1035	2024-12-24 23:18:42:618 CST ACK_ack: 43701 LOSS
467	2024-12-24 23:18:45:016 CST *Re: DATA_seq: 43601 NO_ACK	1036	2024-12-24 23:18:45:523 CST *Re: ACK_ack: 43701
468	2024-12-24 23:18:45:016 CST *Re: DATA_seq: 43501 NO_ACK	1037	2024-12-24 23:18:46:144 CST ACK_ack: 44701
469	2024-12-24 23:18:45:016 CST *Re: DATA_seq: 43401 NO_ACK		
470	2024-12-24 23:18:45:016 CST *Re: DATA_seq: 43301 NO_ACK		
471	2024-12-24 23:18:45:016 CST *Re: DATA_seq: 43201 NO_ACK		
472	2024-12-24 23:18:45:016 CST *Re: DATA_seq: 43101 NO_ACK		
473	2024-12-24 23:18:45:016 CST *Re: DATA_seq: 43001 NO_ACK		
474	2024-12-24 23:18:45:016 CST *Re: DATA_seq: 42901 NO_ACK		

图 45 Sender Log

图 46 Receiver Log

ACK 出错，超时重传整个窗口。

综上，对于 Reno 的关注点实现了以下几点：

- ❑ 检查 Log 文件，当出现错误（包含数据包出错、数据包或确认报丢包）时，发方在 3s 后超时重传当前窗口内的所有未收到确认的包；超时重传的包包含发送端已经收到确认的包，基本 TCP 要点实现正确。
- ❑ 发送过程出现慢开始、加法增大；出现超时重传后，进入快恢复阶段，不进入慢开始。
- ❑ 接收方在传输出错时使用 AckFlag 判断收到包的类型，对于按序或者重发的包延迟 500ms 累积确认，其余（乱序）包立即回复 ACK。
- ❑ 发送方使用 lastAckCount 计数连续收到 3 个相同的 ACK，快重传后重置窗口大小为 ssthresh 进入快恢复阶段。

二、实验评估

1. 未完成关键困难

所有的关键困难都已经解决，实现了 TCP 的基本功能和 Tahoe、Reno 的拥塞控制。

2. 迭代开发

使用 Git 进行版本管理，从 RDT 1.0 开始逐个迭代开发，并创建不同分支存放对应代码、修复后续问题。好处是每一个时间点的代码都可以倒回去查看，而且像 TCP 可以直接从 GBN 和 SR 分支中合并代码，十分方便。迭代开发也有利于同学们由浅入深学习 TCP 发展，更好理解不同时间 TCP 协议的差异和进步。

main Merge branch 'TCP-Reno'	hongir03 9739abb Today at 21:34
TCP-Reno (TCP Reno)(feat) visualize	hongir03 97780f0 Today at 21:33
Merge branch 'TCP-Tahoe' into TCP-Reno	hongir03 c3776ca Today at 21:27
Update README.md	Alorng Hong a614fd4 Yesterday at 21:29
(docs) Update README.md	hongir03 ae2ff47 Dec 17, 2024 at 22:09
[TCP Reno] tested	hongir03 1950813 Dec 17, 2024 at 22:06
[TCP Reno][fix] wrong log	hongir03 ba3653d Dec 17, 2024 at 22:04
[TCP Reno][fix] wrong log	hongir03 eb26c6c Dec 17, 2024 at 21:59
[TCP Reno] tested	hongir03 4f8b367 Dec 17, 2024 at 21:54
[TCP Reno] impl fast recovery	hongir03 3030599 Dec 17, 2024 at 21:54
[TCP Tahoe] tested	hongir03 83a7458 Dec 17, 2024 at 21:47
[TCP Tahoe] remove nextAck, use double cwnd instead	hongir03 17a2f0b Dec 17, 2024 at 21:34
[TCP Tahoe] add sender logger, tested	hongir03 177a63f Dec 17, 2024 at 19:30
[TCP Tahoe] add sender logger	hongir03 0519608 Dec 17, 2024 at 19:16
[TCP Tahoe] tested	hongir03 965db76 Dec 17, 2024 at 19:15
[TCP Tahoe] remove unused println sentence	hongir03 bda4eac Dec 17, 2024 at 19:15
[TCP Tahoe] rewrite SenderWindow, using LinkedBlockingDeque instead of normal array	hongir03 ab5488b Dec 17, 2024 at 19:14
[TCP Tahoe] new construct function of SenderElem	hongir03 k0na80b Dec 17, 2024 at 19:11
TCP [TCP] well we just need to adjust sendWindow then	hongir03 ca87a4a Today at 19:07
[TCP][fix] fast resend	hongir03 5070c54 Today at 18:53
[TCP] backported, formatted, tested	hongir03 fda7a28 Today at 17:05
Merge remote-tracking branch 'origin/TCP' into TCP	hongir03 16600dd Dec 17, 2024 at 16:40
[TCP] tested	hongir03 bc27a9a Dec 17, 2024 at 16:40
[TCP] reformat & refactor	hongir03 75f97ce Dec 17, 2024 at 16:40
[TCP] add quick resend	hongir03 2a6dc18 Dec 17, 2024 at 16:39
Merge remote-tracking branch 'origin/main'	hongir03 d4f017a Dec 17, 2024 at 16:00
[GBN][fix]	hongir03 662396b Dec 17, 2024 at 16:00
[GBN] done and tested	hongir03 7630445 Dec 17, 2024 at 15:06
Dec-BackN [GBN][fix] consistency	hongir03 94cd0cb Today at 18:56
[GBN] formatted, tested	hongir03 388a3c5 Today at 18:21
[GBN] add base class WindowElem, tested	hongir03 83fa799 Today at 16:59
[GBN] tested	hongir03 58232d3 Dec 17, 2024 at 16:47
[GBN] Send a full window one time	hongir03 966f564 Dec 17, 2024 at 16:46
[GBN] Delayed & accumulated ack response	hongir03 38ca24c Dec 17, 2024 at 16:46
[GBN] Change SenderElem timer to SenderWindow timer	hongir03 7096a8b Dec 17, 2024 at 16:46

图 47 提交记录

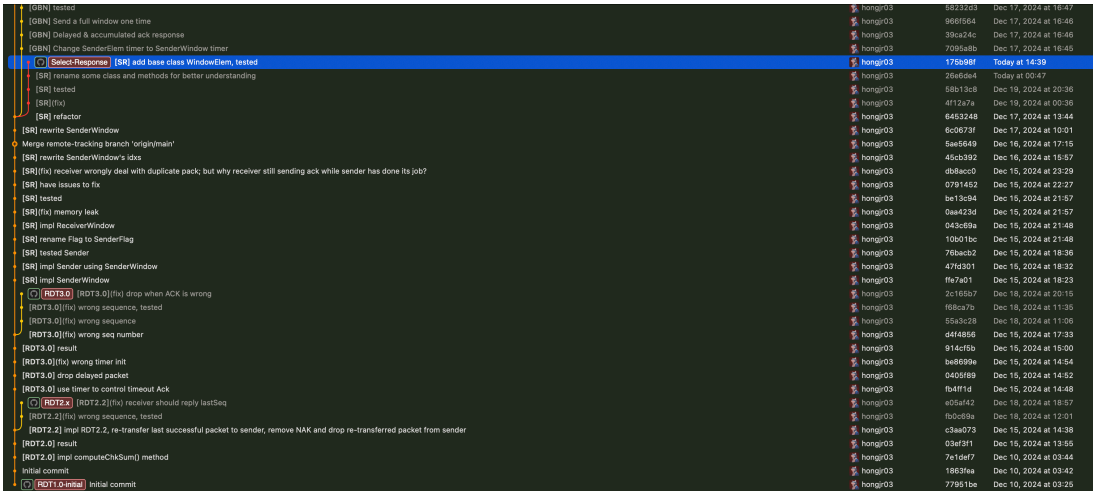


图 48 提交记录

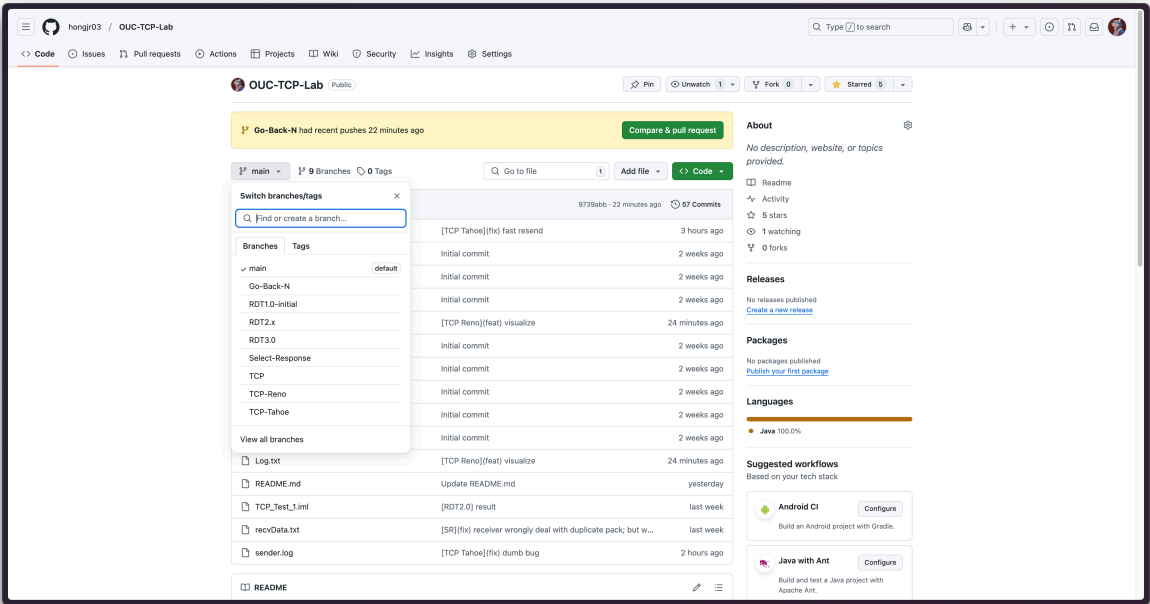


图 49 GitHub 仓库和分支

3. 已解决主要问题

实验过程中时常忘记协议的细节，粗略实现后在写实验报告分析 Log 时经常一边写一边改。

实验过程中有提到，是使用一个窗口一起发送还是逐个分组发送实现的问题。在意义上，我觉得两个都没问题。快重传机制一开始在我的设计策略下显得并不是完成得很好，本应该在连续收到 3 个相同的 ACK 时重传接收方期望的数据包，但在这过程中因为一次发送一个窗口的数据包，如果左沿的包丢失了，则在接下来一整个窗口的 ACK 都会是重复的，导致发送方会收到很多重复的 ACK。尽管快重传机制被触发了，但它重传的数量远大于 1 个。

394	2024-12-21 17:50:20:124 CST DATA_seq: 34601 LOSS NO_ACK	1655
395	2024-12-21 17:50:20:124 CST DATA_seq: 34701 NO_ACK	1656
396	2024-12-21 17:50:20:124 CST DATA_seq: 34801 NO_ACK	1657
397	2024-12-21 17:50:20:124 CST DATA_seq: 34901 NO_ACK	1658
398	2024-12-21 17:50:20:124 CST DATA_seq: 35001 NO_ACK	1659
399	2024-12-21 17:50:20:124 CST DATA_seq: 35101 NO_ACK	1660
400	2024-12-21 17:50:20:125 CST DATA_seq: 35201 NO_ACK	1661
401	2024-12-21 17:50:20:125 CST DATA_seq: 35301 NO_ACK	1662
402	2024-12-21 17:50:20:125 CST DATA_seq: 35401 NO_ACK	1663
403	2024-12-21 17:50:20:125 CST DATA_seq: 35501 NO_ACK	1664
404	2024-12-21 17:50:20:125 CST DATA_seq: 35601 NO_ACK	1665
405	2024-12-21 17:50:20:125 CST DATA_seq: 35701 ACKed	1666
406	2024-12-21 17:50:20:134 CST *Re: DATA_seq: 34601 NO_ACK	1667
407	2024-12-21 17:50:20:134 CST *Re: DATA_seq: 34601 NO_ACK	1668
408	2024-12-21 17:50:20:135 CST *Re: DATA_seq: 34601 NO_ACK	1669
409	2024-12-21 17:50:20:135 CST *Re: DATA_seq: 34601 NO_ACK	1670
410	2024-12-21 17:50:20:136 CST *Re: DATA_seq: 34601 NO_ACK	1671
411	2024-12-21 17:50:20:136 CST *Re: DATA_seq: 34601 NO_ACK	1672
412	2024-12-21 17:50:20:137 CST *Re: DATA_seq: 34601 NO_ACK	1673
413	2024-12-21 17:50:20:138 CST *Re: DATA_seq: 34601 NO_ACK	1674
414	2024-12-21 17:50:20:140 CST *Re: DATA_seq: 34601 NO_ACK	1675
415	2024-12-21 17:50:20:142 CST *Re: DATA_seq: 34601 NO_ACK	1676
416	2024-12-21 17:50:20:148 CST DATA_seq: 35801 ACKed	1677

图 50 Sender Log

2024-12-21 17:50:20:130 CST ACK_ack: 34501
2024-12-21 17:50:20:131 CST ACK_ack: 34501
2024-12-21 17:50:20:133 CST ACK_ack: 34501
2024-12-21 17:50:20:133 CST ACK_ack: 34501
2024-12-21 17:50:20:134 CST ACK_ack: 34501
2024-12-21 17:50:20:134 CST ACK_ack: 34501
2024-12-21 17:50:20:135 CST ACK_ack: 34501
2024-12-21 17:50:20:136 CST ACK_ack: 34501
2024-12-21 17:50:20:136 CST ACK_ack: 34501
2024-12-21 17:50:20:137 CST ACK_ack: 34501
2024-12-21 17:50:20:138 CST ACK_ack: 34501
2024-12-21 17:50:20:138 CST ACK_ack: 34501
2024-12-21 17:50:20:139 CST ACK_ack: 35701
2024-12-21 17:50:20:142 CST ACK_ack: 35701
2024-12-21 17:50:20:143 CST ACK_ack: 35701
2024-12-21 17:50:20:144 CST ACK_ack: 35701
2024-12-21 17:50:20:144 CST ACK_ack: 35701
2024-12-21 17:50:20:145 CST ACK_ack: 35701
2024-12-21 17:50:20:145 CST ACK_ack: 35701
2024-12-21 17:50:20:146 CST ACK_ack: 35701
2024-12-21 17:50:20:146 CST ACK_ack: 35701
2024-12-21 17:50:20:147 CST ACK_ack: 35701
2024-12-21 17:50:20:149 CST ACK_ack: 35801

图 51 Receiver Log

要避免这个问题应该还是要避免整个窗口一起发送的策略，所以继续修改代码中 TCP_Sender 的 `rdt_send` 方法，改为不用等到窗口满就发包：

```

1    // ...
2
3    if (window.isFull()) {
4        //窗口满，等待窗口滑动
5        flag = WindowFlag.FULL.ordinal();
6        // 发送窗口中的数据
7        // window.sendWindow();
8    }
9
10   while (flag == WindowFlag.FULL.ordinal()) {
11       //等待窗口滑动
12   }
13
14   try {
15       window.pushPacket(tcpPack.clone());
16   } catch (CloneNotSupportedException e) {
17       throw new RuntimeException(e);
18   }
19   window.sendWindow();
20
21   // ...
22

```

而原来的 `sendWindow` 方法并没有被删除，在超时重传时还是会用到，我重命名为 `resendWindow` 方法。

还有一个困扰了有段时间的问题是处理 Receiver 发回的 ACK。以下是修改前后的代码：

```
1 public void ackPacket(int seq) {
2     int idx = getIdIdx(base);
3     while (
4         window[idx]
5             .getPacket()
6             .getTcpH()
7             .getTh_seq() <= seq
8             && !window[idx].isAked()
9             && base < rear
10    ) {
11        window[idx].ackPacket();
12        base++;
13        idx = getIdIdx(base);
14    }
15    resetTimer();
16 }
17 }
```

代码 1 修改前

```
1 public void ackPacket(int seq) {
2     for (int i = base; i != rear; i++) {
3         int idx = getIdIdx(i);
4         if (
5             window[idx]
6                 .getPacket()
7                 .getTcpH()
8                 .getTh_seq() <= seq
9                 && !window[idx].isAked()
10        ) {
11            window[idx].ackPacket();
12            window[idx].resetElem();
13            base++;
14            resetTimer();
15        }
16    }
17 }
```

代码 2 修改后

主要问题是出在高亮区域比 `base < rear` 更先判断，导致程序可能出现空指针错误。想到这样也不易读，于是我就改成了右边这样的写法。

4. 实验系统建议

1. 提醒同学们实验的时候不要开 **Clash** 的 **TUN 模式**！每次都觉得很疑惑，怎么启动了发不出包呢？又或者是每次写着写着怎么突然发不出包了？后来发现是 Clash 的 TUN 模式开着，导致包全都被拦截了。
2. 日志里可以把两方的 Log 合在一起，这样可以更好地看到两方的交互。